



UNIVERSIDAD DEL BÍO-BÍO

CIMUBB
LABORATORIO DE SISTEMAS
AUTOMATIZADOS DE PRODUCCION
Facultad de Ingeniería

Proyecto: Asistencia digital

Integrantes :
Marcelo Paz
Bastián Rodríguez

Supervisor :
Luis Vera

28 de febrero de 2025

Índice de contenidos

1	Introducción	4
1.1	Situación actual	4
1.2	Objetivos del proyecto	4
1.2.1	Objetivo general	4
1.2.2	Objetivos específicos	4
2	Análisis y Diseño	5
2.1	Diseño lógico	5
2.1.1	Modelo de entidad-relación	5
2.1.2	Modelo relacional	5
2.1.3	Diagrama de contexto	6
2.1.4	Diagrama de nivel superior	6
2.1.5	Procedimientos administrativos	7
2.2	Diseño físico	8
2.2.1	Arquitectura del sistema	8
3	Documentación de Hardware	9
3.1	Raspberry Pi 4	9
3.2	Pantalla táctil de 7 pulgadas [Se desconoce el modelo de pantalla]	9
3.3	Logitech HD Webcam C270	10
3.4	Sensor QR	10
4	Documentación desarrollo del Software	11
4.1	Etapas 0	11
4.2	Etapas 1	11
4.3	Etapas 2	11
4.4	Etapas 3 (Sin pruebas)	11
4.5	Etapas 4 (Sin pruebas)	12
5	Implementación	12
5.1	Software consultor	12
5.1.1	Librerías	12
5.1.2	Estructura de archivos y directorios	13
5.1.3	Puesta en marcha (MANUAL)	14
5.1.4	Puesta en marcha (AUTOMÁTICA)	19
5.1.5	Capturas de la interfaz	19
5.1.6	Código	23
5.2	Software registro de asistencia	58
5.2.1	Librerías	58
6	Repositorio de GitHub	59

Índice de figuras

1	Modelo de entidad-relación	5
2	Modelo relacional	5
3	Diagrama de contexto	6
4	Diagrama de nivel superior	6
5	Procedimientos administrativos	7
6	Arquitectura del sistema	8
7	Raspberry Pi 4 [1]	9
8	Pantalla táctil de 7 pulgadas	9
9	Logitech HD Webcam C270 [2]	10
10	Sensor QR GM811 [3]	10
11	Requerimientos software consultor	12
12	Estructura de archivos y directorios	13
13	Generate Random Strings and Numbers [4]	18
14	Interfaz del home	19
15	Interfaz del login	20
16	Interfaz del dashboard	20
17	Interfaz del profile	21
18	Interfaz del register admin	21
19	Interfaz del register user	22
20	Componente Navbar	22
21	Componente Notify	22
22	Archivo auth.py Parte I	23
23	Archivo auth.py Parte II	23
24	Archivo auth.py Parte III	24
25	Archivo security.py	24
26	Archivo database.py	25
27	Archivo query.py Parte I	25
28	Archivo query.py Parte II	26
29	Archivo query.py Parte III	27
30	Archivo query.py Parte IV	27
31	Archivo query.py Parte V	28
32	Archivo main.py Parte I	28
33	Archivo main.py Parte II	29
34	Archivo main.py Parte III	29
35	Archivo main.py Parte IV	29
36	Archivo main.py Parte V	30
37	Archivo main.py Parte VI	30
38	Archivo main.py Parte VII	31
39	Archivo main.py Parte VIII	32
40	Archivo main.py Parte IX	32
41	Archivo main.py Parte X	33
42	Archivo main.py Parte XI	34
43	Archivo main.py Parte XII	34
44	Archivo main.py Parte XIII	35
45	Archivo main.py Parte XIV	35



46	Archivo main.py Parte XV	36
47	Archivo main.py Parte XVI	36
48	Archivo main.py Parte XVII	37
49	Archivo base.html	37
50	Archivo navbar.html	38
51	Archivo navbar.js Parte I	39
52	Archivo navbar.js Parte II	40
53	Archivo dashboard.html Parte I	40
54	Archivo dashboard.html Parte II	41
55	Archivo dashboard.html Parte III	41
56	Archivo dashboard.html Parte IV	41
57	Archivo dashboard.html Parte V	42
58	Archivo dashboard.html Parte VI	42
59	Archivo dashboard.html Parte VII	42
60	Archivo dashboard.html Parte VIII	42
61	Archivo dashboard.html Parte IX	43
62	Archivo dashboard.html Parte X	43
63	Archivo dashboard.html Parte XI	44
64	Archivo dashboard.html Parte XII	44
65	Archivo charts.js Parte I	45
66	Archivo charts.js Parte II	45
67	Archivo charts.js Parte III	46
68	Archivo charts.js Parte IV	47
69	Archivo data.js Parte I	48
70	Archivo data.js Parte II	49
71	Archivo data.js Parte III	50
72	Archivo data.js Parte IV	51
73	Archivo data.js Parte V	52
74	Archivo data.js Parte VI	53
75	Archivo data.js Parte VII	53
76	Archivo data.js Parte VIII	54
77	Archivo data.js Parte IX	54
78	Archivo pagination.js Parte I	54
79	Archivo pagination.js Parte II	55
80	Archivo pagination.js Parte III	55
81	Archivo pagination.js Parte IV	56
82	Archivo pagination.js Parte V	56
83	Archivo pagination.js Parte VI	57
84	Archivo pagination.js Parte VII	57
85	Archivo sidebar.js	58
86	Repositorio del proyecto [5]	59

Índice de tablas

1. Introducción

1.1. Situación actual

Respecto al estudio realizado en el establecimiento, se observó un gran flujo de personas que hacen ingreso al Laboratorio utilizando un registro de asistencia manual. Algunas de estas personas son trabajadores de la institución, estudiantes en prácticas o invitados. Existen ocasiones especiales, como exposiciones de maquetas o presentaciones de nuevas tecnologías en desarrollo, en la cual ingresa personal externo a la institución.

1.2. Objetivos del proyecto

1.2.1. Objetivo general

- Construir un sistema de asistencia digital que permita conocer el ingreso y salida de los individuos dentro del laboratorio.
- Visualizar los datos registrados en un dashboard con información sobre el uso del espacio y las actividades principales.

1.2.2. Objetivos específicos

- Recepción de datos de individuos.
- Validación de datos utilizando un API proporcionada por la Universidad del Bío-Bío.
- Almacenamiento de datos registrados.
- Visualización de datos registrados.

2. Análisis y Diseño

2.1. Diseño lógico

2.1.1. Modelo de entidad-relación

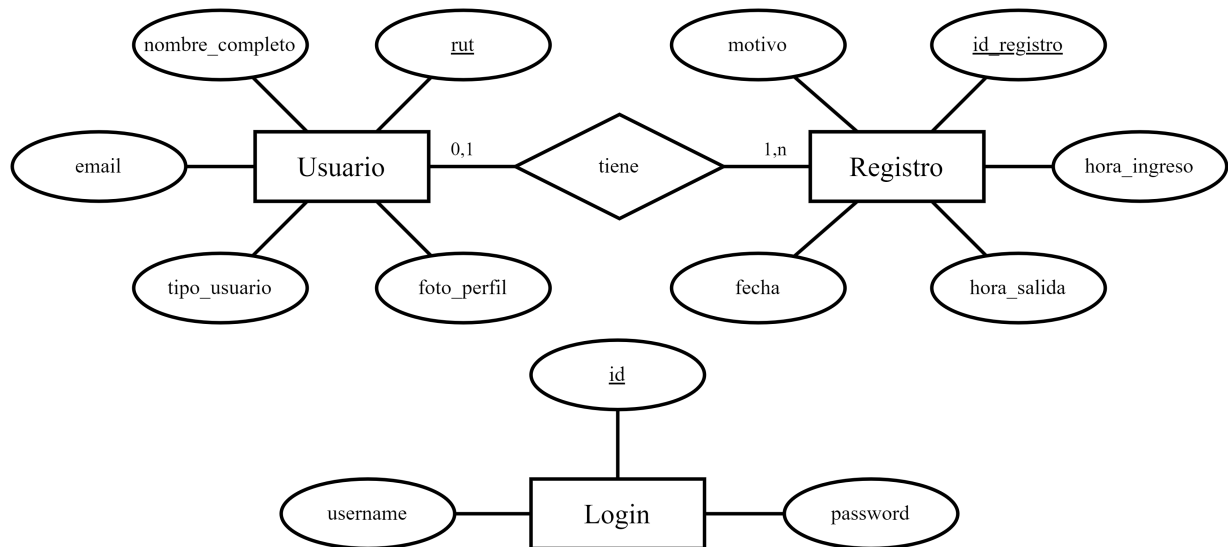


Figura 1: Modelo de entidad-relación

2.1.2. Modelo relacional

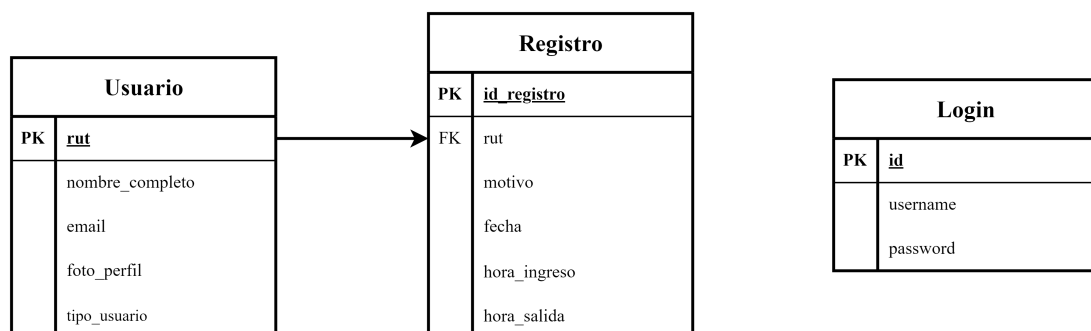


Figura 2: Modelo relacional

2.1.3. Diagrama de contexto

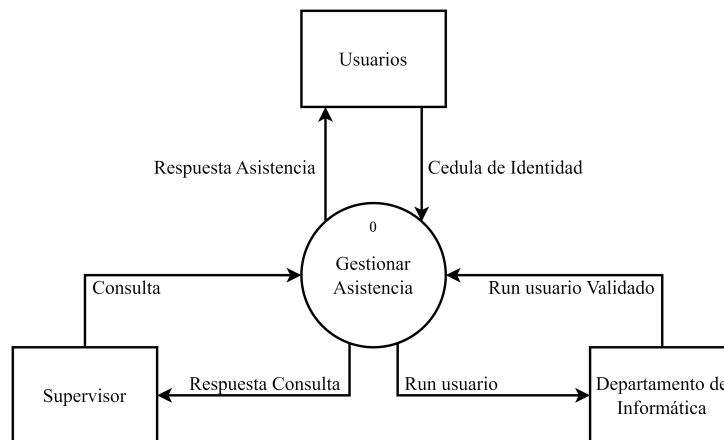


Figura 3: Diagrama de contexto

2.1.4. Diagrama de nivel superior

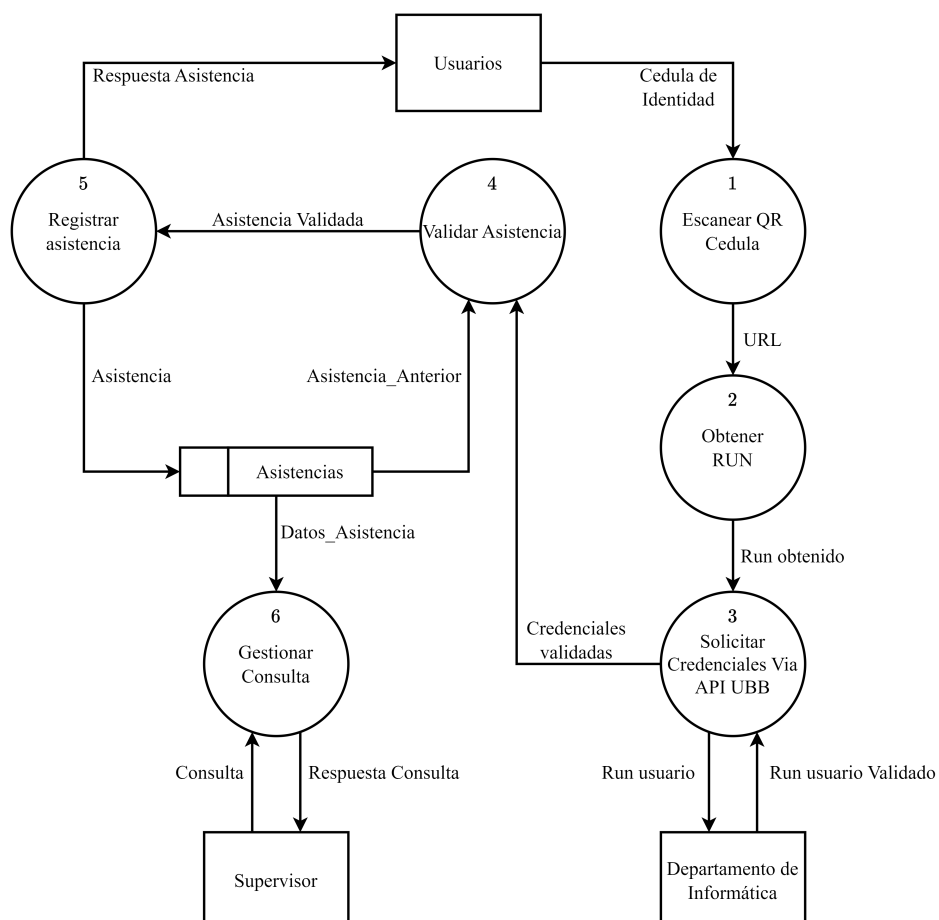


Figura 4: Diagrama de nivel superior

2.1.5. Procedimientos administrativos

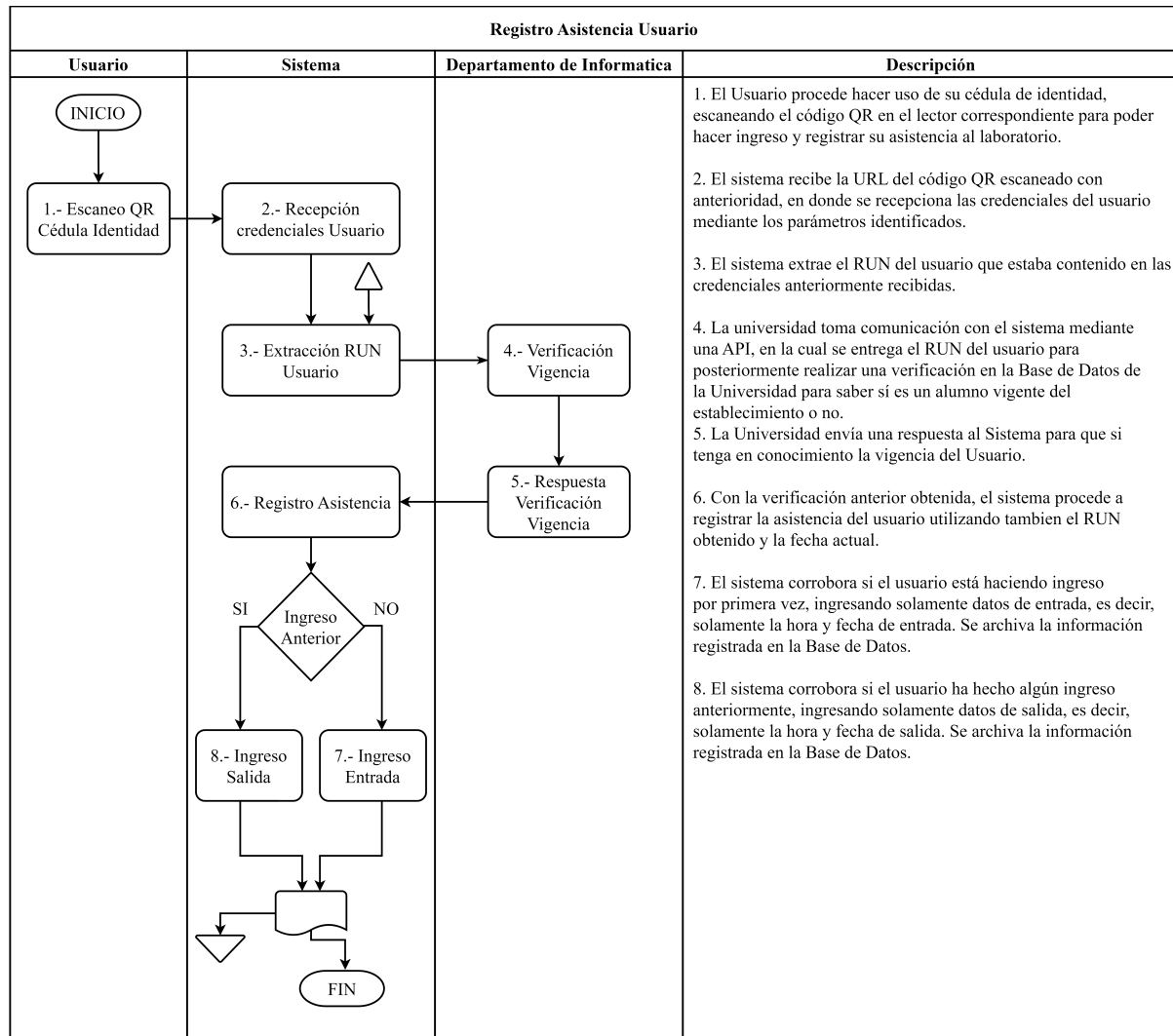


Figura 5: Procedimientos administrativos

2.2. Diseño físico

2.2.1. Arquitectura del sistema

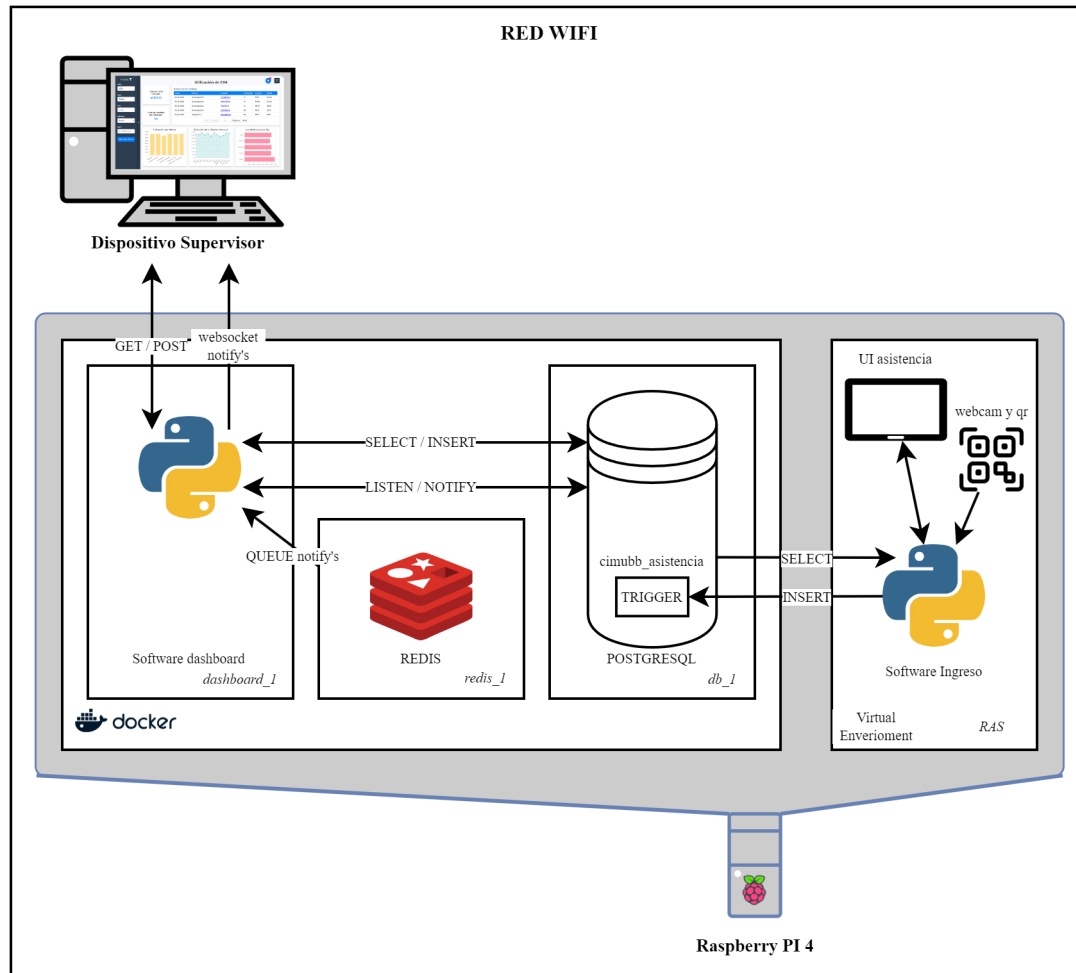


Figura 6: Arquitectura del sistema

Se decidió utilizar Docker para facilitar la implementación del software. En resumen, dentro de la raspberry tendremos 3 imagenes de Docker y un entorno virtual para el correcto funcionamiento de la aplicación de registro:

- **dashboard_1:** Se utiliza una imagen de python13 slim con la aplicación que controla el servicio web para la supervisión de los registros.
- **redis_1:** Se utiliza una imagen oficial de redis para el guardado de notificaciones temporales, pues si no tenemos activo un WebSocket se pierden las notificaciones, por lo que el usar Redis de forma temporal nos resuelve este problema al utilizar una Queue.
- **db_1:** Se utiliza una imagen oficial de postgres:17 para guardar nuestros datos y montar inicialmente la configuración.
- **RAS:** La aplicación encargada de mostrar una GUI para el registro de asistencias del laboratorio.

3. Documentación de Hardware

3.1. Raspberry Pi 4

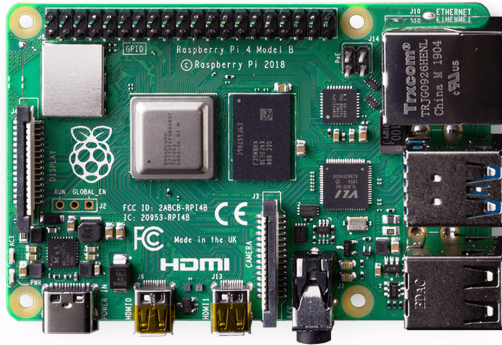


Figura 7: Raspberry Pi 4 [1]

La Raspberry Pi 4 Model B de 8GB de RAM es utilizada en el proyecto por su capacidad para manejar aplicaciones intensivas en procesamiento y datos, su conectividad avanzada, que incluye Wi-Fi y Ethernet, asegura la comunicación eficiente con servidores o sistemas remotos, facilitando el monitoreo y notificación del ingreso de personal al laboratorio.

3.2. Pantalla táctil de 7 pulgadas [Se desconoce el modelo de pantalla]



Figura 8: Pantalla táctil de 7 pulgadas

La pantalla táctil de 7 pulgadas es utilizada para mostrar el software de registro de asistencia, permitiendo a los usuarios ingresar su información personal y registrar su ingreso al laboratorio.

3.3. Logitech HD Webcam C270



Figura 9: Logitech HD Webcam C270 [2]

La Logitech HD Webcam C270 es utilizada para capturar la foto de perfil de las personas que se registran en el sistema.

3.4. Sensor QR



Figura 10: Sensor QR GM811 [3]

El sensor QR GM811 es utilizado para leer los códigos QR de las cédulas de identidad, de las personas que ingresan al laboratorio, permitiendo un registro rápido y eficiente de la asistencia.

Lo más importante de este sensor es que tiene un modo de POS (Point of Sale) que permite leer códigos QR de manera rápida y eficiente, lo que lo hace ideal para el proyecto.

4. Documentación desarrollo del Software

4.1. Etapa 0

Se elaboro de un informe para la asignatura Sistemas de Información, en el cual se desarrolló un prototipo de un sistema de asistencia digital para el Laboratorio de Sistemas Automatizados de Producción de la Universidad del Bío-Bío. El software inicialmente fue desarrollado de forma rudimentaria, pues lo importante del informe era llevar a cabo una idea, definir ciertos aspectos y realizar los diagramas de contexto, nivel superior y de procedimientos administrativos.

4.2. Etapa 1

En esta etapa se utilizo la idea de la Etapa 0 para desarrollarlo en la practica profesional II dentro del Laboratorio de Sistemas Automatizados de Producción de la Universidad del Bío-Bío.

Se utilizo inicialmente un dispositivo Windows con una cámara web para el registro de asistencia, el software analizaba la imagen con opencv y detectaba el código QR de la cédula de identidad, luego se enviaba la información a una base de datos en PostgreSQL.

4.3. Etapa 2

En esta etapa se utilizo una Raspberry Pi 4 con una cámara web, se utilizo el SO Raspbian y el mismo software de la etapa 1.

Se observo que la Raspberry Pi 4 tenia problemas al procesar la imagen de la cámara web, tanto en la detección del código QR como en la velocidad de actualización de la imagen en la interfaz gráfica. Al intentar resolver el problema, se crearon más problemáticas, pues se intento implementar la pantalla tactil que se comunicaba por GPIO, lo que genero problemáticas en la comunicación, dejando la pantalla en negro, por lo que se tuvo que reinstalar el SO.

Finalmente se decidió cambiar el software de registro de asistencia y utilizar un Sensor QR.

4.4. Etapa 3 (Sin pruebas)

Se desarrollo desde cero, el software de registro de asistencia, implementando el sensor QR. Lo malo es que mientras se descubria el funcionamiento del sensor, se configuro a un modo que solo aceptaba configuracion por comunicacion Serial, pero luego no era reconocido por ningun dispositivo. Lo que nos retraso en nuestro limitado tiempo.

El supervisor nos recomendo hacer algunos avances sin realizar pruebas para entregar al menos una idea del software a implementar, para completar la practica profesional II.

4.5. Etapa 4 (Sin pruebas)

Se desarrollo una aplicación web para la visualización de los datos, para ello se utilizo Python con FastAPI. Para el acceso a esta aplicación, el supervisor debe acceder a la IP de las raspberry por el puerto 8000, la aplicación cuenta con un ingreso por PWT, para la seguridad mínima del sistema.

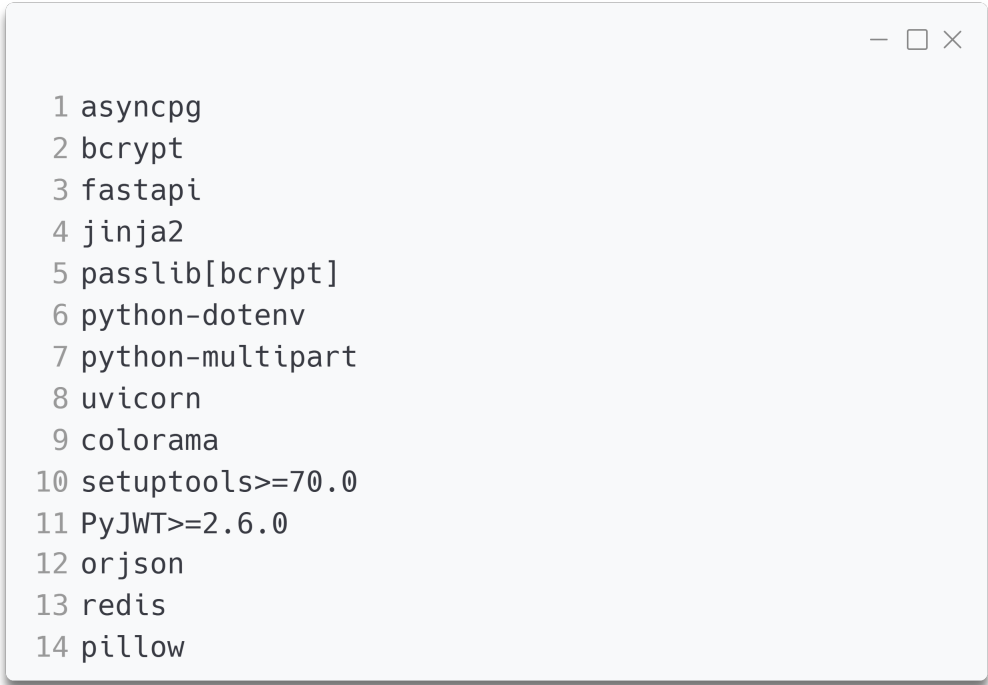
Durante el desarrollo se fueron reformulando ciertas ideas, lo que llevo a algunas modificaciones de la BD, como por ejemplo la capacidad de notificar de un nuevo ingreso, la actualización automatica de la hora de salida si es que una persona no hubiera marcado y la actualización automatica de la zona horaria para invierno/verano.

Al final de esta etapa se decidió utilizar Docker para contener los diferentes módulos del SW, aparte de su facil implementación, pues con el docker compose se puede crear facilmente una DB segura y compacta.

5. Implementación

5.1. Software consultor

5.1.1. Librerías



```
1 asyncpg
2 bcrypt
3 fastapi
4 jinja2
5 passlib[bcrypt]
6 python-dotenv
7 python-multipart
8 uvicorn
9 colorama
10 setuptools>=70.0
11 PyJWT>=2.6.0
12 orjson
13 redis
14 pillow
```

Figura 11: Requerimientos software consultor

5.1.2. Estructura de archivos y directorios

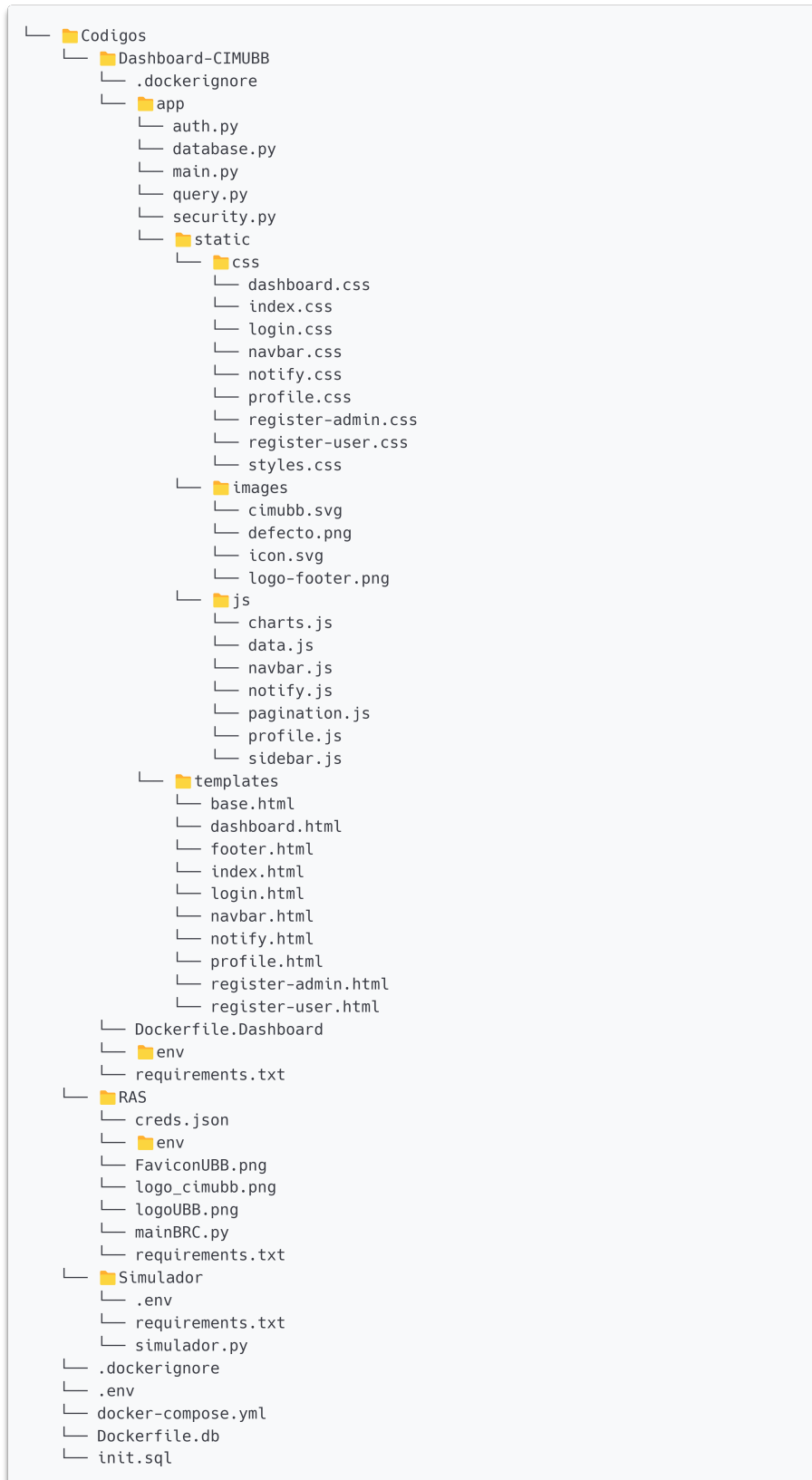


Figura 12: Estructura de archivos y directorios

5.1.3. Puesta en marcha (MANUAL)

WINDOWS

1. Clonamos el repositorio:

```
git clone https://github.com/CIMUBB/Asistencia-CIMUBB.git
```

2. Nos movemos a la carpeta del proyecto:

```
cd .\Codigos\
```

3. Utilizando PgAdmin 4, DBeaver o la conexión por terminal, creamos una base de datos en PostgreSQL con el nombre *cimubb_asistencia*.

- a) Crear la base de datos con el comando:

```
CREATE DATABASE cimubb_asistencia;
```

- b) Nos conectamos a la BD creada:

```
\c cimubb_asistencia
```

Forma automática:

- c) Ejecutamos el archivo init.sql, utilizando la ruta del archivo. Como cron no existe para windows las tareas de marcar salida y actualizar zona horaria no funcionarían:

```
\i ruta\al\repositorio\Asistencia-CIMUBB\Codigos\init.sql
```

Forma manual:

- d) Creamos las tablas:

```
CREATE TABLE Usuario (  
    rut VARCHAR(10) PRIMARY KEY NOT NULL,  
    nombre_completo VARCHAR(50) DEFAULT '',  
    email VARCHAR(50) DEFAULT '',  
    tipo_usuario VARCHAR(15) NOT NULL,  
    foto_perfil BYTEA DEFAULT NULL  
);  
  
CREATE TABLE Registro (  
    id_registro VARCHAR(10) PRIMARY KEY NOT NULL,  
    fecha DATE NOT NULL,  
    hora_ingreso TIME NOT NULL,  
    hora_salida TIME DEFAULT NULL,  
    motivo VARCHAR(50) NOT NULL,  
    rut VARCHAR(10) NOT NULL,  
    FOREIGN KEY (rut) REFERENCES Usuario(rut)  
);  
  
CREATE TABLE Login (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR UNIQUE NOT NULL,  
    password VARCHAR NOT NULL  
);
```

- e) Habilitamos la extensión crypto e insertamos el login para el admin:

```
-- Extensión para almacenar contraseñas encriptadas  
CREATE EXTENSION IF NOT EXISTS pgcrypto;  
  
-- Insertar usuario administrador con contraseña encriptada  
INSERT INTO Login (username, password)  
VALUES ('admin', crypt('admin', gen_salt('bf')));
```

***OBS:** Por obvias razones se recomienda cambiar el username y el password del login a la dashboard.

- f) Habilitamos la extensión cron y creamos una función para marcar la salida automática a las 19:00 de los registros que se les olvidó marcar la hora salida:

```
-- Habilitar pg_cron si no existe
CREATE EXTENSION IF NOT EXISTS pg_cron;

-- Crear la función para actualizar la hora de salida automáticamente
CREATE OR REPLACE FUNCTION marcar_salida_automatica()
RETURNS VOID AS $$
BEGIN
    UPDATE Registro
    SET hora_salida = '19:00:00'
    WHERE hora_salida IS NULL AND fecha = CURRENT_DATE;
END;
$$ LANGUAGE plpgsql;

-- Programar la tarea para ejecutarse todos los días a las HH:MM
SELECT cron.schedule(
    'marcar_salida_diaria',
    '00 22 * * *', -- MM HH DD MM DOW (https://crontab.guru/)
    'SELECT marcar_salida_automatica();'
);
```

- g) Creamos una función para habilitar un LISTEN / NOTIFY con un trigger al detectar un ingreso o salida:

```
CREATE OR REPLACE FUNCTION notify_registro_changes()
RETURNS TRIGGER AS $$
DECLARE
    payload TEXT;
BEGIN
    IF TG_OP = 'INSERT' THEN
        payload := NEW.rut || ' ingreso a las ' || TO_CHAR(NEW.hora_ingreso,
        'HH24:MI');
    ELSIF TG_OP = 'UPDATE' AND OLD.hora_salida IS DISTINCT FROM NEW.hora_salida
    THEN
        payload := NEW.rut || ' salio a las ' || TO_CHAR(NEW.hora_salida,
        'HH24:MI');
    ELSE
        RETURN NEW;
    END IF;

    PERFORM pg_notify('notify_channel', payload);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER registro_notify_trigger
AFTER INSERT OR UPDATE ON Registro
FOR EACH ROW
EXECUTE FUNCTION notify_registro_changes();
```

- h) Creamos una función para actualizar la zona horaria, pues postgres trabaja con UTC:

```
CREATE OR REPLACE FUNCTION actualizar_horario_pgcron()
RETURNS VOID AS $$
DECLARE
    diferencia_horaria INT;
    nueva_hora INT;
    nueva_hora_cron INT;
BEGIN
    -- Detectar la diferencia horaria real de Chile con UTC
    SELECT EXTRACT(TIMEZONE_HOUR FROM NOW()) AT TIME ZONE 'America/Santiago') INTO
    diferencia_horaria;

    IF diferencia_horaria = -3 THEN
        nueva_hora := 22; -- Chile UTC-3 (verano)
        nueva_hora_cron := 3; -- Medianoche en Chile en UTC-3
    ELSE
        nueva_hora := 23; -- Chile UTC-4 (invierno)
        nueva_hora_cron := 4; -- Medianoche en Chile en UTC-4
    END IF;

    -- Eliminar el job anterior si existe
    PERFORM cron.unschedule('marcar_salida_diaria');
    PERFORM cron.unschedule('actualizar_horario_diario');

    -- Programar la tarea para actualizar la salida automática
    PERFORM cron.schedule(
        'marcar_salida_diaria',
        format('00 %s * * *', nueva_hora),
        'SELECT marcar_salida_automatica();'
    );

    -- Programar la tarea para actualizar el horario de ejecución
    PERFORM cron.schedule(
        'actualizar_horario_diario',
        format('0 %s * * *', nueva_hora_cron),
        'SELECT actualizar_horario_pgcron();'
    );
END;
$$ LANGUAGE plpgsql;

SELECT cron.schedule(
    'actualizar_horario_diario',
    '0 3 * * *', -- Ejecutar cada día a medianoche UTC CHILE
    'SELECT actualizar_horario_pgcron();'
);
```

*OBS: El paso g) y h) no funcionarían porque CRON no existe para Windows, por lo que estas funciones no aplican, pues el software fue desarrollado para raspberryPi.

4. Volviendo al repositorio nos movemos a la carpeta Dashboard:

```
cd .\Codigos\Dashboard-CIMUBB\
```

5. Creamos un entorno virtual de python y lo activamos:

```
python -m venv env  
.\env\Scripts\activate.bat
```

6. Instalamos las dependencias:

```
pip install -r .\requirements.txt
```

7. Configuramos el archivo .env dentro de la carpeta Dashboard-CIMUBB con las siguientes variables:

```
SECRET_KEY=  
ACCESS_TOKEN_EXPIRE_MINUTES=30  
DB_HOST=  
DB_PORT=5432  
DB_NAME=cimubb_asistencia  
DB_USER=postgres  
DB_PASSWORD=
```

***OBS:** Para la generación del SECRET_KEY utilizaremos la pagina browserling[4]

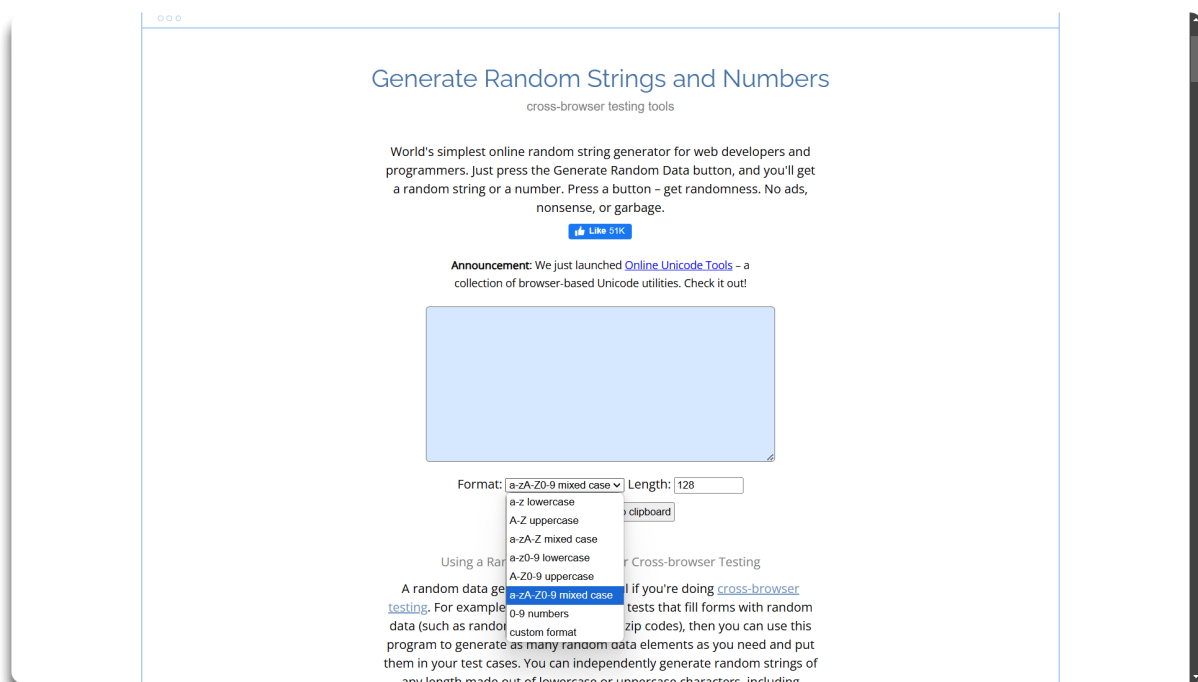


Figura 13: Generate Random Strings and Numbers [4]

8. Para finalizar ejecutamos el servidor de UVICORN:

```
uvicorn main.app:app --host 0.0.0.0 --port 8000 --workers 2
```

5.1.4. Puesta en marcha (AUTOMÁTICA)

DOCKER

1. Teniendo instalado docker, clonamos el repositorio:

```
git clone https://github.com/CIMUBB/Asistencia-CIMUBB.git
```

2. Nos movemos a la carpeta del proyecto:

```
cd .\Codigos\
```

3. Construimos las imagenes utilizando docker compose:

```
docker compose build --no-cache
```

4. Activamos los contenedores:

```
docker compose up -d
```

5.1.5. Capturas de la interfaz

- <http://{IP}:8000/>



Figura 14: Interfaz del home

- <http://{IP}:8000/login>

Asistencia CIM

Iniciar Sesión

Por favor, ingresa tus credenciales

Rut (sin puntos y con guion)

Contraseña

Iniciar Sesión



Concepción: Avda. Collao N° 1202, Casilla 5-C · C.P: 4081112 · Fono: 56 (41) 311 1308

Email: lvera@ubiobio.cl

Figura 15: Interfaz del login

- <http://{IP}:8000/dashboard>

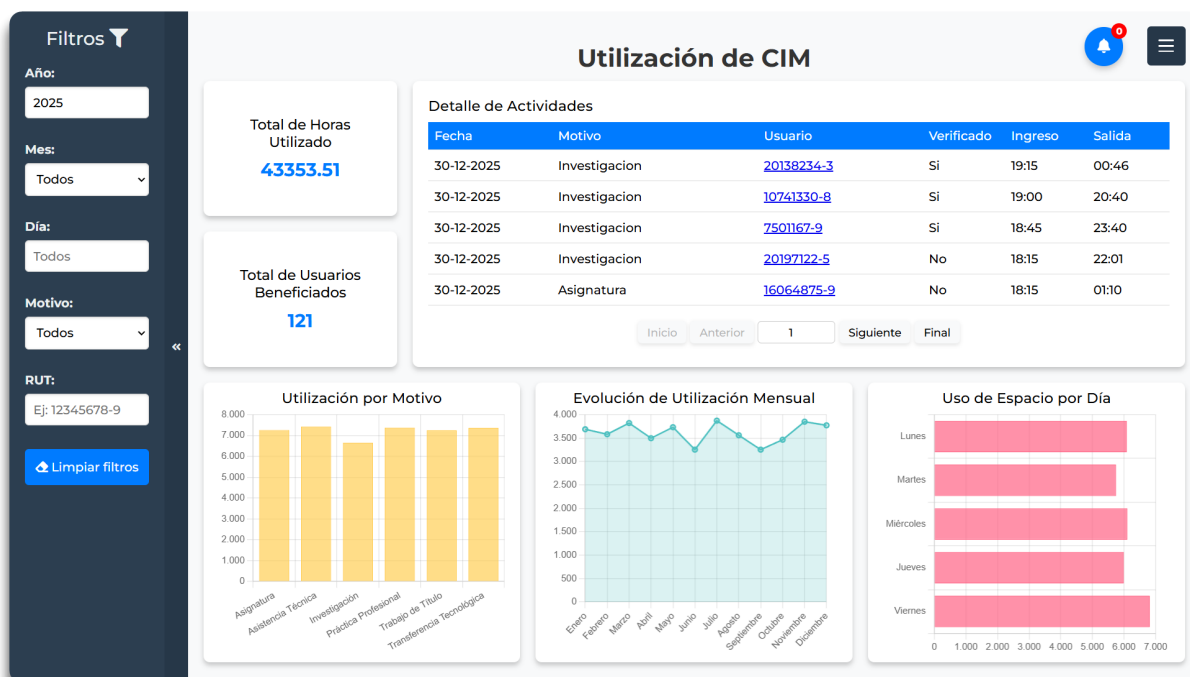


Figura 16: Interfaz del dashboard

- <http://{IP}:8000/profile?rut={RUT}>

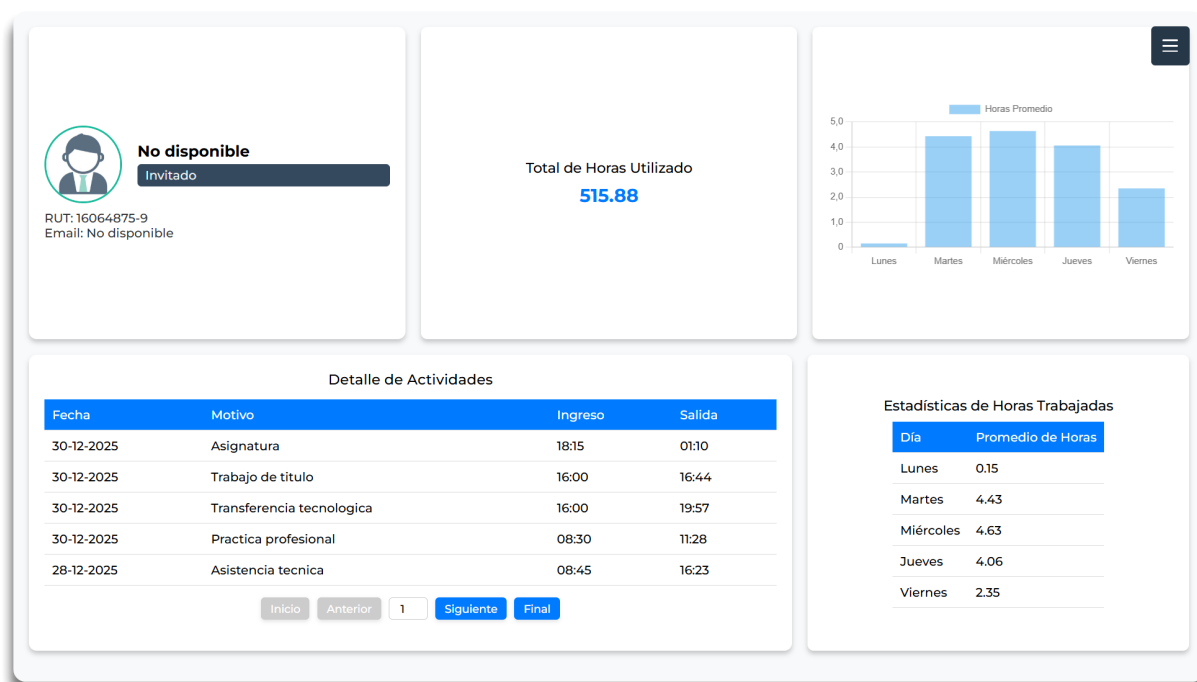
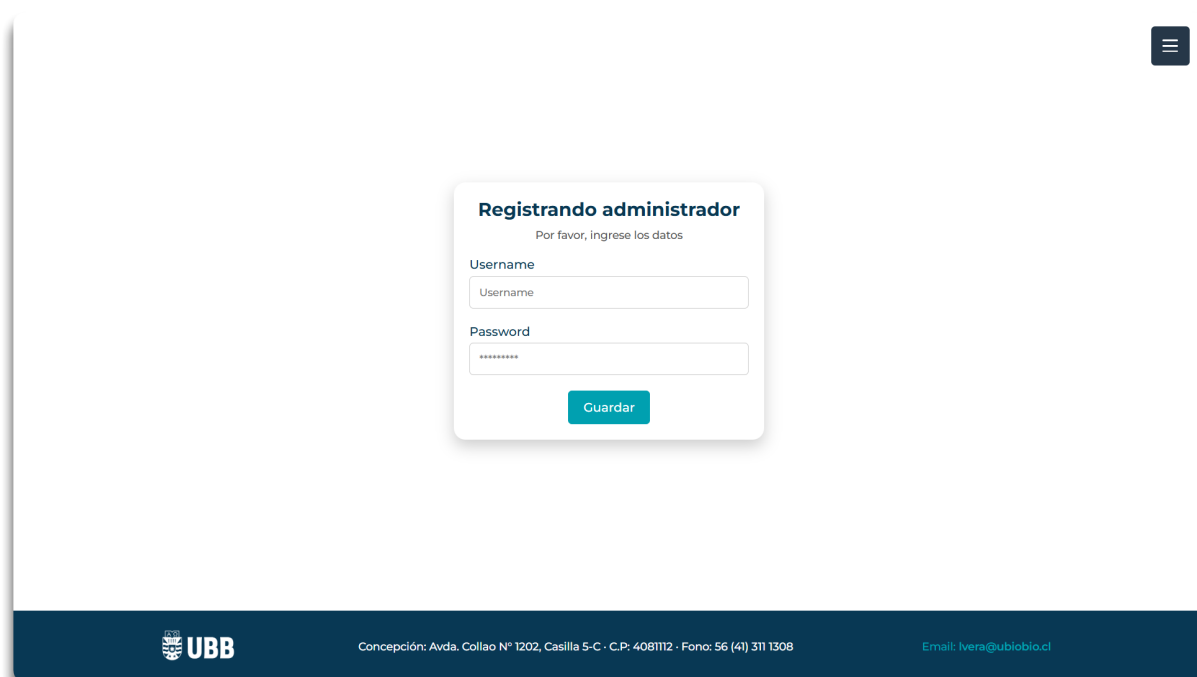


Figura 17: Interfaz del profile

- <http://{IP}:8000/register-admin>



Registrando administrador
Por favor, ingrese los datos

Username
Username

Password

Guardar

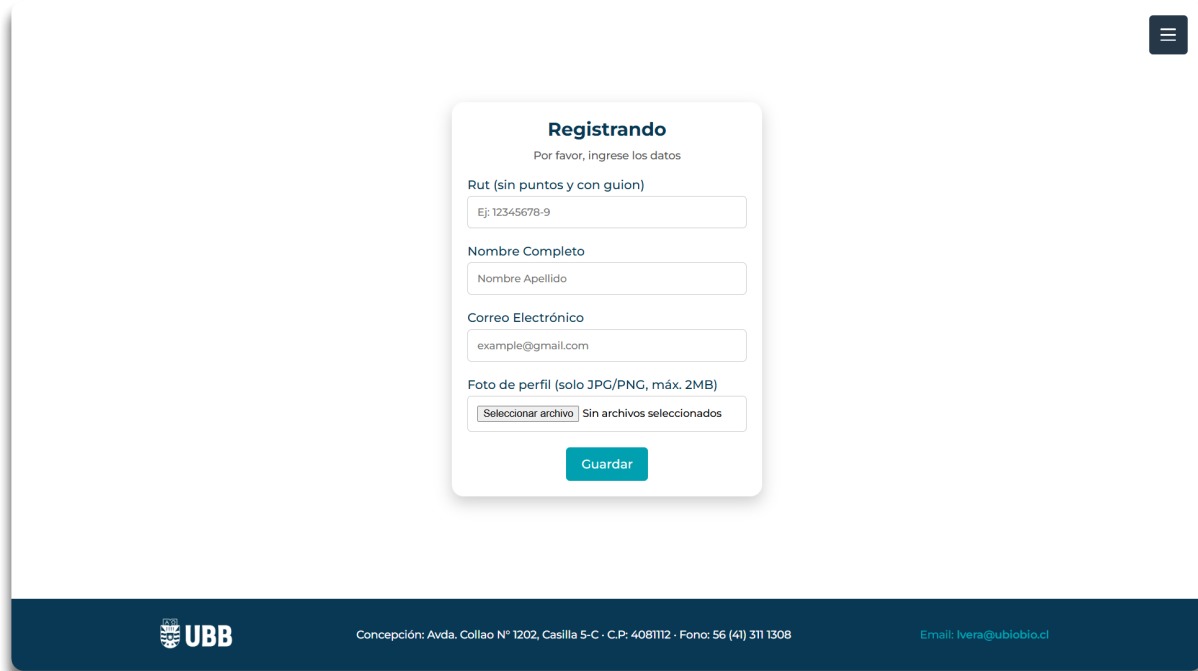
UBB

Concepción: Avda. Collao N° 1202, Casilla 5-C · C.P: 4081112 · Fono: 56 (41) 311 1308

Email: lvera@ubiobio.cl

Figura 18: Interfaz del register admin

- `http://{IP}:8000/register-user`



The screenshot shows a web interface for user registration. At the top right is a hamburger menu icon. The main content is a white card titled "Registrando" with the subtitle "Por favor, ingrese los datos". The form includes the following fields: "Rut (sin puntos y con guion)" with a placeholder "Ej: 12345678-9", "Nombre Completo" with a placeholder "Nombre Apellido", "Correo Electrónico" with a placeholder "example@gmail.com", and "Foto de perfil (solo JPG/PNG, máx. 2MB)" with a file selection button "Seleccionar archivo" and a status "Sin archivos seleccionados". A green "Guardar" button is at the bottom of the card. The footer is a dark blue bar with the UBB logo, contact information for Concepción, and an email address "Email: lvera@ubiobio.cl".

Figura 19: Interfaz del register user

- Componentes Navbar y Notify:

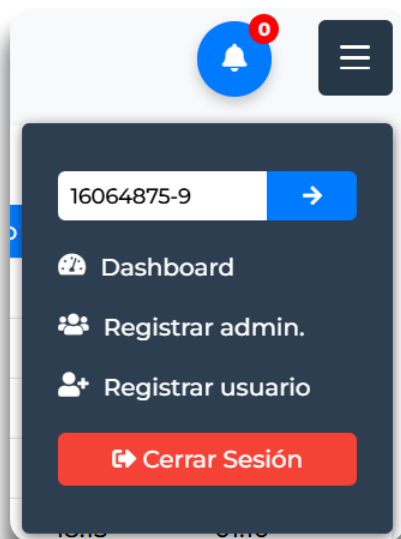


Figura 20: Componente Navbar

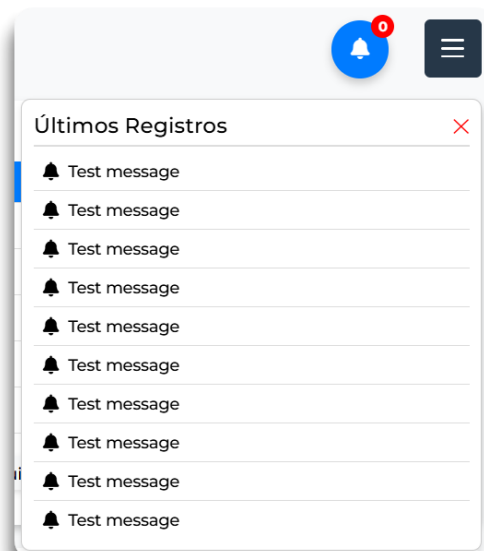


Figura 21: Componente Notify

5.1.6. Código

Por la obvia cantidad de código que engloba todo el proyecto se dejara captura de gran parte del código y se explicara algunas de las partes que al autor le parecen importantes.

1. auth.py:

a) La importación de librerías, la obtención del secreto y la duración del token:

```
1 import os
2 import jwt
3 from datetime import datetime, timedelta, timezone
4 from fastapi import HTTPException
5 from fastapi.security import OAuth2PasswordBearer
6 from dotenv import load_dotenv
7 from colorama import Fore, Style
8
9 load_dotenv()
10
11 SECRET_KEY = os.getenv("SECRET_KEY")
12 if not SECRET_KEY:
13     raise ValueError("ERROR: La variable de entorno 'SECRET_KEY' es obligatoria.")
14
15 ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", 30))
16 ALGORITHM = "HS256"
17
18 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/login")
```

Figura 22: Archivo auth.py Parte I

b) Una función que crea un token JWT para el login del supervisor:

```
20 def create_access_token(data: dict) → str:
21     """
22     Crea un token de acceso JWT con datos y tiempo de expiración.
23     """
24     if "sub" not in data:
25         raise ValueError("El campo 'sub' es obligatorio en los datos del token.")
26
27     to_encode = data.copy()
28     expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
29     to_encode.update({"exp": expire})
30
31     try:
32         encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
33         print(f"{Fore.YELLOW}CIMUBB:\t {Style.RESET_ALL} Token creado exitosamente.")
34         return encoded_jwt
35     except Exception as e:
36         print(f"ERROR: Error al crear el token: {e}")
37         raise ValueError("Error al crear el token JWT.")
```

Figura 23: Archivo auth.py Parte II

- c) Una función que verifica si el token es inválido o a sido experida al pasar los 30 minutos:

```
39 def verify_token(token: str) → dict:
40     """
41     Verifica y decodifica un token JWT. Lanza una excepción si el token es inválido o expirado.
42     """
43     try:
44         payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
45         username: str = payload.get("sub")
46         if username is None:
47             raise HTTPException(status_code=401, detail="Token inválido: falta el campo 'sub'.")
48
49         print(f"{Fore.YELLOW}CIMUBB:\t {Style.RESET_ALL} Token válido para el usuario {Fore.YELLOW}{username}{Style.RESET_ALL}")
50         return payload
51     except jwt.ExpiredSignatureError:
52         print("ERROR: El token ha expirado.")
53         raise HTTPException(status_code=401, detail="Token expirado.")
54     except jwt.InvalidTokenError as e:
55         print(f"ERROR: Error al decodificar el token: {e}")
56         raise HTTPException(status_code=401, detail="Token inválido.")
57
```

Figura 24: Archivo auth.py Parte III

2. security.py:

- a) Se crean 2 funciones que permiten comparar el hash del password del login:

```
1 import bcrypt
2
3 def hash_password(password: str) → str:
4     return bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')
5
6 def verify_password(plain_password: str, hashed_password: str) → bool:
7     return bcrypt.checkpw(plain_password.encode('utf-8'), hashed_password.encode('utf-8'))
```

Figura 25: Archivo security.py

3. database.py:

- a) Importación de librerías, creación de objeto Database y conexión por piscinas, lo que mejora significativamente la cantidad simultánea de consultas:

```
1 import os
2 import asyncpg
3 from dotenv import load_dotenv
4
5 load_dotenv()
6
7 class Database:
8     def __init__(self):
9         self.pool = None
10
11     async def connect(self):
12         self.pool = await asyncpg.create_pool(
13             host=os.getenv("DB_HOST", "db"),
14             port=os.getenv("DB_PORT", 5432),
15             database=os.getenv("DB_NAME", "cimubb_asistencia"),
16             user=os.getenv("DB_USER", "postgres"),
17             password=os.getenv("DB_PASSWORD", "password"),
18             min_size=1,
19             max_size=4,
20             ssl=False
21         )
22
23         async with self.pool.acquire() as conn:
24             encoding = await conn.fetchval("SHOW server_encoding;")
25             if encoding != "UTF8":
26                 raise ValueError(f"La BD no está en UTF-8, {encoding}")
27     async def disconnect(self):
28         if self.pool:
29             await self.pool.close()
30
31 db = Database()
```

Figura 26: Archivo database.py

4. query.py:

- a) Importación de librerías:

```
1 from datetime import datetime, timedelta
2 from app.database import db
3 from app.security import verify_password
4 import asyncio
5 from colorama import Fore, Style
```

Figura 27: Archivo query.py Parte I

- b) Función principal que obtiene todos los datos a mostrar en el dashboard según los parametros:

```
7 # Función para obtener los registros filtrados desde la base de datos
8 async def get_filtered_data(year=None, month=None, day=None, motivo=None, rut=None):
9     async with db.pool.acquire() as connection:
10         query = """
11             SELECT r.id_registro, r.fecha, r.hora_ingreso, r.hora_salida,
12                    r.motivo, r.rut, u.tipo_usuario
13             FROM registro r
14             JOIN usuario u ON r.rut = u.rut
15             WHERE TRUE
16         """
17
18         conditions = []
19         params = []
20
21         placeholder_index = 1
22
23         if year:
24             conditions.append(f"EXTRACT(YEAR FROM r.fecha) = ${placeholder_index}")
25             params.append(int(year))
26             placeholder_index += 1
27
28         if month and month != "all":
29             conditions.append(f"EXTRACT(MONTH FROM r.fecha) = ${placeholder_index}")
30             params.append(int(month))
31             placeholder_index += 1
32
33         if day is not None and day != "":
34             conditions.append(f"EXTRACT(DAY FROM r.fecha) = ${placeholder_index}")
35             params.append(int(day))
36             placeholder_index += 1
37
38         valid_motivos = [
39             "asignatura", "asistencia tecnica", "investigacion",
40             "practica", "trabajo de titulo", "transferencia tecnologica"
41         ]
42         if motivo and motivo != "all":
43             if motivo.lower() in valid_motivos:
44                 conditions.append(f"r.motivo ILIKE ${placeholder_index}")
45                 params.append(motivo.lower())
46                 placeholder_index += 1
47             else:
48                 raise ValueError(f"Motivo inválido: {motivo}")
49
50         if rut and rut != "":
51             conditions.append(f"r.rut LIKE ${placeholder_index}")
52             params.append(f"{rut}%") # Coincidir con RUT que comiencen con el valor proporcionado
53             placeholder_index += 1
54
55         if conditions:
56             query += " AND " + " AND ".join(conditions)
57
58         query += " ORDER BY r.fecha DESC, r.hora_ingreso DESC"
59
60         registros = await connection.fetch(query, *params)
61
62         return [
63             {
64                 "id_registro": registro["id_registro"],
65                 "fecha": registro["fecha"].isoformat() if registro["fecha"] else None,
66                 "hora_ingreso": str(registro["hora_ingreso"]),
67                 "hora_salida": str(registro["hora_salida"]),
68                 "motivo": registro["motivo"],
69                 "verificado": "No" if registro["tipo_usuario"] == "invitado" else "Si",
70                 "rut": registro["rut"]
71             }
72             for registro in registros
73         ]
```

Figura 28: Archivo query.py Parte II

c) Función que obtiene todos los registros de un rut entregado por parametros:

```
75 async def get_all_data_by_rut(rut):
76     async with db.pool.acquire() as connection:
77         query = """
78             SELECT r.fecha, r.hora_ingreso, r.hora_salida, r.motivo, r.rut, u.nombre_completo, u.email, u.foto_perfil, u.tipo_usuario
79             FROM registro r
80             JOIN usuario u ON r.rut = u.rut
81             WHERE r.rut = $1
82             ORDER BY r.fecha DESC, r.hora_ingreso DESC
83         """
84         registros = await connection.fetch(query, rut)
85         return [
86             {
87                 "fecha": registro["fecha"].isoformat() if registro["fecha"] else None,
88                 "hora_ingreso": str(registro["hora_ingreso"]),
89                 "hora_salida": str(registro["hora_salida"]),
90                 "motivo": registro["motivo"],
91                 "rut": registro["rut"],
92                 "nombre_completo": registro["nombre_completo"] or "No disponible",
93                 "email": registro["email"] or "No disponible",
94                 "tipo_usuario": registro["tipo_usuario"],
95                 "foto_perfil": registro["foto_perfil"]
96             }
97             for registro in registros
98         ]
```

Figura 29: Archivo query.py Parte III

d) Función que verifica la existencia de un login, si no existe el objetivo es mostrarle al supervisor que las credenciales de ingreso son incorrectas:

```
100 async def authenticate_user(username: str, password: str):
101     async with db.pool.acquire() as connection:
102         query = """
103             SELECT * FROM login WHERE username = $1;
104         """
105         user = await connection.fetchrow(query, username)
106         if user and verify_password(password, user["password"]):
107             return user
108         return None
```

Figura 30: Archivo query.py Parte IV

- e) Función que se encarga de obtener los mensajes de la DB, esto para notificar en la aplicación web de ingresos o salidas según corresponda:

```
110 async def listen_postgres(queue: asyncio.Queue):
111     """Escucha notificaciones en PostgreSQL y las coloca en la cola de mensajes."""
112     async with db.pool.acquire() as connection:
113         await connection.execute("SET client_encoding = 'UTF8';")
114
115         def callback(conn, pid, channel, payload):
116             """Callback para manejar las notificaciones de PostgreSQL."""
117             try:
118                 queue.put_nowait(payload) # Agregar evento a la cola
119             except Exception as e:
120                 print(f"{Fore.YELLOW}CIMUBB:{Style.RESET_ALL}\t Error decodificando mensaje de PostgreSQL: {e}")
121
122         await connection.add_listener("notify_channel", callback)
123
124         try:
125             while True:
126                 await asyncio.sleep(1)
127         except asyncio.CancelledError:
128             print(f"{Fore.YELLOW}CIMUBB:{Style.RESET_ALL}\t PostgreSQL Listener cancelado correctamente.")
129
130         finally:
131             await connection.remove_listener("notify_channel", callback)
132             print(f"{Fore.YELLOW}CIMUBB:{Style.RESET_ALL}\t PostgreSQL Listener desconectado.")
```

Figura 31: Archivo query.py Parte V

5. main.py:

- a) Importación de librerías «inserte calavera»:

```
1 from fastapi import FastAPI, Query, Request, Depends, HTTPException, Form, WebSocket, WebSocketDisconnect, File, UploadFile
2 from fastapi.exceptions import RequestValidationError
3 from fastapi.responses import HTMLResponse, RedirectResponse, JSONResponse, ORJSONResponse, Response
4 from fastapi.security import OAuth2PasswordRequestForm
5 from fastapi.staticfiles import StaticFiles
6 from fastapi.templating import Jinja2Templates
7 from starlette.websockets import WebSocketState
8 from starlette.responses import RedirectResponse
9 from contextlib import asynccontextmanager
10 from colorama import Fore, Style
11 from datetime import datetime
12 import redis
13 import asyncio
14 import asyncpg
15 from PIL import Image
16 import io
17 from urllib.parse import urlencode
18
19 # Dependencias propias
20 from app.query import (
21     authenticate_user,
22     get_filtered_data,
23     get_all_data_by_rut,
24     listen_postgres
25 )
26 from app.auth import (
27     create_access_token,
28     verify_token
29 )
30 from app.security import hash_password
31 from app.database import db
```

Figura 32: Archivo main.py Parte I

b) Conexión con redis para manejar la cola de notificaciones:

```
33 redis_client = redis.StrictRedis(host="localhost", port=6379, db=0, decode_responses=True)
34
35 message_queue = asyncio.Queue()
36 websockets = set()
```

Figura 33: Archivo main.py Parte II

c) Función que decide si envía las notificaciones directamente a los WebSockets activos, de lo contrario los guardara temporalmente en redis:

```
38 async def process_events():
39     """Procesa eventos y los envía a WebSockets activos o los guarda en Redis."""
40     while True:
41         payload = await message_queue.get() # Obtener el evento desde PostgreSQL
42         print(f"enviando a {len(websockets)}")
43         print(f"PostgreSQL emitió: {payload}")
44
45         # Limpiar WebSockets desconectados antes de enviar
46         disconnected_ws = set()
47         for ws in websockets:
48             try:
49                 await ws.send_text(payload)
50             except Exception:
51                 disconnected_ws.add(ws)
52
53         # Eliminar WebSockets desconectados
54         websockets.difference_update(disconnected_ws)
55
56         # Guardar en Redis si no hay WebSockets activos
57         if not websockets:
58             redis_client.rpush("pending_notifications", payload)
```

Figura 34: Archivo main.py Parte III

d) Función que valida el formato de la imagen pasada por parametros en el endpoint de register-user:

```
60 async def validate_image(photo: UploadFile):
61     try:
62         image_bytes = await photo.read()
63         image = Image.open(io.BytesIO(image_bytes))
64
65         # Validar formato de imagen
66         if image.format not in ["JPEG", "PNG"]:
67             raise HTTPException(status_code=400, detail="Formato inválido. Solo se permiten imágenes JPG o PNG.")
68
69         return image_bytes
70
71     except Exception:
72         raise HTTPException(status_code=400, detail="El archivo no es una imagen válida.")
```

Figura 35: Archivo main.py Parte IV

- e) Control del ciclo de vida del software, que permite controlar y detener funciones si es que se detiene el SW:

```
74 # Ciclo de vida de la aplicación
75 @asynccontextmanager
76 async def lifespan(app: FastAPI):
77     await db.connect()
78     print(f"{Fore.YELLOW}{40 * '='} Aplicación iniciada {40 * '='}{Style.RESET_ALL}")
79     print(f"{Fore.YELLOW}CIMUBB:{Style.RESET_ALL}\t http://127.0.0.1:8000")
80     postgres_task = asyncio.create_task(listen_postgres(message_queue))
81     events_task = asyncio.create_task(process_events())
82     yield
83
84     postgres_task.cancel()
85     events_task.cancel()
86     await db.disconnect()
87     print(f"{Fore.RED}{40 * '='} Aplicación cerrada {40 * '='}{Style.RESET_ALL}")
88
89 # Inicialización de FastAPI
90 app = FastAPI(debug=False, lifespan=lifespan)
91
92 # Configuración de plantillas y archivos estáticos
93 templates = Jinja2Templates(directory="app/templates")
94 app.mount("/static", StaticFiles(directory="app/static"), name="static")
```

Figura 36: Archivo main.py Parte V

- f) Importación de CORS y definición de rutas protegidas:

```
96 from fastapi.middleware.cors import CORSMiddleware
97
98 app.add_middleware(
99     CORSMiddleware,
100     allow_credentials=True,
101     allow_methods=["GET", "POST", "DELETE"],
102     allow_headers=["*"],
103 )
104
105 PROTECTED_ROUTES = ("/dashboard", "/profile", "/register-admin", "/register-user")
```

Figura 37: Archivo main.py Parte VI

- g) Protección del middleware, para permitir el acceso a URL específicas solo si esta validado su JWT:

```
107 @app.middleware("http")
108 async def auth_middleware(request: Request, call_next):
109     path = request.url.path
110     if path in PROTECTED_ROUTES:
111         access_token = request.cookies.get("access_token")
112
113         if not access_token or not access_token.startswith("Bearer "):
114             response = RedirectResponse(url="/login", status_code=303)
115             response.set_cookie(
116                 key="access_token",
117                 value="",
118                 httponly=True,
119                 max_age=0,
120                 expires=0,
121                 path="/",
122             )
123             return response
124
125         try:
126             token = access_token[len("Bearer "):]
127             verify_token(token)
128         except Exception as e: # Captura cualquier error en la verificación
129             response = RedirectResponse(url="/login", status_code=303)
130             response.set_cookie(
131                 key="access_token",
132                 value="",
133                 httponly=True,
134                 max_age=0,
135                 expires=0,
136                 path="/",
137             )
138             return response
139
140     response = await call_next(request)
141     if path in PROTECTED_ROUTES:
142         response.headers.update({
143             "Cache-Control": "no-store, no-cache, must-revalidate, max-age=0",
144             "Pragma": "no-cache",
145             "Expires": "0"
146         })
147     return response
```

Figura 38: Archivo main.py Parte VII

h) Endpoint que entrega páginas html estáticas:

```
150 @app.get("/", response_class=HTMLResponse)
151 async def index_page(request: Request):
152     return templates.TemplateResponse("index.html", {"request": request, "page": "index"})
153
154 @app.get("/login", response_class=HTMLResponse)
155 async def login_page(request: Request):
156     return templates.TemplateResponse("login.html", {"request": request, "page": "login"})
157
158 @app.get("/dashboard", response_class=HTMLResponse)
159 async def dashboard_page(request: Request):
160     return templates.TemplateResponse("dashboard.html", {"request": request, "page": "dashboard"})
161
162 @app.get("/register-user", response_class=HTMLResponse)
163 async def register_user_page(request: Request):
164     return templates.TemplateResponse("register-user.html", {"request": request, "page": "register-user"})
165
166 @app.get("/register-admin", response_class=HTMLResponse)
167 async def register_admin_page(request: Request):
168     return templates.TemplateResponse("register-admin.html", {"request": request, "page": "register-admin"})
```

Figura 39: Archivo main.py Parte VIII

i) Endpoint para facilitar el acceso a imágenes de perfil que pueda poseer un usuario:

```
170 @app.get("/profile/photo/{rut}")
171 async def get_profile_photo(rut: str):
172     """
173     Devuelve la foto de perfil del usuario como una imagen binaria.
174     Si no tiene foto, devuelve la imagen por defecto.
175     """
176     async with db.pool.acquire() as connection:
177         query = "SELECT foto_perfil FROM usuario WHERE rut = $"
178         row = await connection.fetchrow(query, rut)
179
180         if row and row["foto_perfil"]:
181             return Response(content=row["foto_perfil"], media_type="image/png")
182
183         return Response(content=open("app/static/images/defecto.png", "rb").read(), media_type="image/png")
```

Figura 40: Archivo main.py Parte IX

- j) Endpoint para mostrar la página web dinámica profile, la que mediante un rut muestra todos los registros de ese usuario:

```
185 @app.get("/profile", response_class=ORJSONResponse)
186 async def profile_page(
187     request: Request,
188     rut: str = Query(..., min_length=9, max_length=12, regex=r"^\d{7,8}-?[0-9kk]?$",
189 ):
190     try:
191         registros = await get_all_data_by_rut(rut)
192
193         if not registros:
194             raise HTTPException(status_code=404, detail="Usuario no encontrado")
195
196         usuario = registros[0]
197
198         if isinstance(usuario, asyncpg.Record):
199             usuario = dict(usuario)
200
201         # Convertir cualquier campo 'bytes' en 'str'
202         for key, value in usuario.items():
203             if isinstance(value, bytes):
204                 try:
205                     usuario[key] = value.decode("utf-8", errors="ignore")
206                 except Exception:
207                     usuario[key] = str(value)
208
209         foto_perfil_url = f"/profile/photo/{usuario['rut']}" if usuario.get("foto_perfil") else "/static/images/defecto.png"
210
211         registros_limpios = []
212         for record in registros:
213             record_dict = dict(record)
214             for key, value in record_dict.items():
215                 if isinstance(value, bytes):
216                     try:
217                         record_dict[key] = value.decode("utf-8", errors="ignore")
218                     except Exception:
219                         record_dict[key] = str(value)
220             registros_limpios.append(record_dict)
221
222         response_data = {
223             "rut": usuario["rut"],
224             "nombre_completo": usuario["nombre_completo"],
225             "email": usuario["email"],
226             "tipo_usuario": usuario["tipo_usuario"],
227             "foto_perfil": foto_perfil_url,
228             "registros": registros_limpios
229         }
230
231         if "text/html" in request.headers.get("accept", ""):
232             return templates.TemplateResponse("profile.html", {
233                 "request": request,
234                 "page": "profile",
235                 **response_data
236             })
237         else:
238             return ORJSONResponse(content=response_data)
239
240     except Exception as e:
241         print("Error:", str(e))
242         raise HTTPException(status_code=500, detail=f"Error interno del servidor: {str(e)}")
```

Figura 41: Archivo main.py Parte X

- k) Endpoint para iniciar sesión, el cual validará que el usuario exista y que la contraseña sea correcta. Luego le entregará un token JWT para poder ingresar a las páginas protegidas:

```
244 @app.post("/login")
245 async def login(request: Request, form_data: OAuth2PasswordRequestForm = Depends()):
246     user = await authenticate_user(form_data.username, form_data.password)
247     if not user:
248         return templates.TemplateResponse(
249             "login.html", {"request": request, "error": "Credenciales inválidas", "page": "login"}
250         )
251
252     token = create_access_token({"sub": user["username"]})
253     response = RedirectResponse(url="/dashboard", status_code=303)
254     response.set_cookie(
255         key="access_token",
256         value=f"Bearer {token}",
257         httponly=True,
258         max_age=1800,
259         path="/",
260         secure=True,
261         samesite="Lax"
262     )
263     return response
```

Figura 42: Archivo main.py Parte XI

- l) Endpoint para cerrar sesión y eliminar el token de las cookies:

```
265 @app.post("/logout")
266 async def logout():
267     response = RedirectResponse(url="/login", status_code=303)
268     response.delete_cookie("access_token", path="/")
269     print(f"{Fore.YELLOW}CIMUBB:\t {Style.RESET_ALL} Token eliminado correctamente.")
270     return response
```

Figura 43: Archivo main.py Parte XII

m) Endpoint para entregar los datos y notificaciones en el dashboard:

```
272 @app.get("/api/dashboard")
273 async def dashboard_data(
274     year: int = Query(None, ge=2000, le=datetime.now().year),
275     month: str = Query(None, regex="^(0?[1-9]|1[0-2])$"),
276     day: int = Query(None, ge=1, le=31),
277     motivo: str = Query(None, min_length=3, max_length=100),
278     rut: str = Query(None, min_length=1, max_length=12, regex=r"^\d{1,8}-?[0-9kk]?") # Permite búsquedas parciales
279 ):
280     registros = await get_filtered_data(year, month, day, motivo, rut)
281
282     pending_notifications = []
283     while redis_client.llen("pending_notifications") > 0:
284         pending_notifications.append(redis_client.lpop("pending_notifications"))
285
286     return ORJSONResponse(content={
287         "registros": [dict(record) for record in registros],
288         "notificaciones": pending_notifications
289     })
```

Figura 44: Archivo main.py Parte XIII

n) Endpoint para registrar un nuevo usuario desde la aplicación web:

```
291 @app.post("/register-user")
292 async def register_user(
293     rut: str = Form(..., regex=r"^\d{7,8}-?[0-9kk]"),
294     nombre_completo: str = Form(""),
295     email: str = Form(""),
296     photo: UploadFile = File(None)
297 ):
298     async with db.pool.acquire() as conn:
299         query_check = "SELECT rut FROM Usuario WHERE rut = $1"
300         existing_user = await conn.fetchval(query_check, rut)
301
302         if existing_user:
303             raise HTTPException(status_code=400, detail="El usuario ya existe")
304
305         if nombre_completo and email and photo:
306             tipo_usuario = "registrado"
307             photo_bytes = await validate_image(photo)
308         else:
309             tipo_usuario = "invitado"
310             nombre_completo = ""
311             email = ""
312             photo_bytes = None
313
314         query = """
315             INSERT INTO Usuario (rut, nombre_completo, email, tipo_usuario, foto_perfil)
316             VALUES ($1, $2, $3, $4, $5)
317         """
318         await conn.execute(query, rut, nombre_completo, email, tipo_usuario, photo_bytes)
319
320     message = urlencode({"message": "Usuario registrado exitosamente"})
321     return RedirectResponse(url=f"/dashboard?{message}", status_code=303)
```

Figura 45: Archivo main.py Parte XIV

- ñ) Endpoint para registrar un nuevo administrador (LOGIN) desde la aplicación web:

```
323 @app.post("/register-admin")
324 async def register_admin(
325     username: str = Form(...),
326     password: str = Form(...),
327 ):
328     async with db.pool.acquire() as conn:
329         query_check = "SELECT id FROM Login WHERE username = $1"
330         existing_admin = await conn.fetchval(query_check, username)
331
332         if existing_admin:
333             raise HTTPException(status_code=400, detail="El administrador ya existe")
334
335         hashed_password = hash_password(password)
336
337         query = """
338             INSERT INTO Login (username, password)
339             VALUES ($1, $2)
340         """
341         await conn.execute(query, username, hashed_password)
342
343         message = urlencode({"message": "Administrador registrado exitosamente"})
344         return RedirectResponse(url=f"/dashboard?{message}", status_code=303)
```

Figura 46: Archivo main.py Parte XV

- o) Endpoint para agregar un nuevo WebSocket a la escucha, en el que se entregaran las notificaciones que se encuentren en cola y evitar el término de la comunicación:

```
346 @app.websocket("/ws/notify")
347 async def websocket_endpoint(websocket: WebSocket):
348     await websocket.accept()
349     websockets.add(websocket)
350     print(f"{Fore.YELLOW}CIMUBB:{Style.RESET_ALL}\t WebSocket conectado. Total activos: {len(websockets)}")
351     try:
352         while redis_client.llen("pending_notifications") > 0:
353             message = await redis_client.lpop("pending_notifications")
354             if message:
355                 await websocket.send_text(message.decode("utf-8"))
356
357         while True:
358             try:
359                 data = await websocket.receive_text()
360                 if data == 'ping':
361                     continue
362             except WebSocketDisconnect:
363                 print(f"{Fore.YELLOW}CIMUBB:{Style.RESET_ALL}\t Cliente desconectado.")
364                 break
365             except Exception as e:
366                 print(f"{Fore.YELLOW}CIMUBB:{Style.RESET_ALL}\t Error en WebSocket: {e}")
367                 break
368
369     except Exception as e:
370         print(f"{Fore.YELLOW}CIMUBB:{Style.RESET_ALL}\t WebSocket cerrado inesperadamente: {e}")
371
372     finally:
373         websockets.discard(websocket)
374         print(f"{Fore.YELLOW}CIMUBB:{Style.RESET_ALL}\t WebSocket eliminado. Total activos: {len(websockets)}")
```

Figura 47: Archivo main.py Parte XVI

- p) Excepción que se encarga de mejorar el envío de errores para la fácil detección del campo que puede estar generando problemas:

```
376 @app.exception_handler(RequestValidationError)
377 async def validation_exception_handler(request: Request, exc: RequestValidationError):
378     errors = []
379     for error in exc.errors():
380         errors.append({"field": error["loc"][-1], "message": error["msg"]})
381     return ORJSONResponse(content={"errors": errors}, status_code=400)
```

Figura 48: Archivo main.py Parte XVII

6. base.html:

- a) En resumen es la base para el renderizado, se utiliza jinja2 el cual mediante definición de bloques se puede ir ensamblando html más pequeños, en esta base agregamos scripts y los iconos generales, reconocemos la etiqueta de la página y agregamos los estilos específicos:

```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5     <meta charset="UTF-8" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <link rel="icon" type="image/svg+xml" href="/static/images/icon.svg" sizes="any">
8     <link rel="stylesheet" href="/static/css/styles.css" />
9     <link rel="stylesheet" href="/static/css/navbar.css" />
10    <link rel="stylesheet" href="/static/css/notify.css" />
11    <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.4/css/all.min.css" rel="stylesheet">
12    <link href="https://fonts.googleapis.com/css2?family=Montserrat:wght@400;500;600;700&display=swap" rel="stylesheet">
13    {% if page == "index" %}
14    <link rel="stylesheet" href="/static/css/index.css" />
15    {% elif page == "login" %}
16    <link rel="stylesheet" href="/static/css/login.css" />
17    {% elif page == "dashboard" %}
18    <link rel="stylesheet" href="/static/css/dashboard.css?v=1.0" />
19    {% elif page == "table" %}
20    <link rel="stylesheet" href="/static/css/table.css" />
21    {% elif page == "register-user" %}
22    <link rel="stylesheet" href="/static/css/register-user.css" />
23    {% elif page == "register-admin" %}
24    <link rel="stylesheet" href="/static/css/register-admin.css" />
25    {% elif page == "profile" %}
26    <link rel="stylesheet" href="/static/css/profile.css" />
27    {% endif %}
28    <title>{% block title %}{% endblock %}</title>
29 </head>
30
31 <body>
32     {% block header %}
33     {% endblock %}
34     <main>
35         {% block content %}{% endblock %}
36     </main>
37     {% if page != "dashboard" and page != "profile" %}
38     {% include "footer.html" %}
39     {% endif %}
40 </body>
41 </html>
```

Figura 49: Archivo base.html

7. navbar.html:

a) Este componente contiene las referencias a las secciones de la aplicación web:

```
1 <nav class="navbar">
2   <button class="hamburger" aria-label="Menu" aria-expanded="false" aria-controls="navbar-menu">
3     <span></span>
4     <span></span>
5     <span></span>
6   </button>
7   <div class="navbar-menu" id="navbar-menu">
8     <div class="navbar-item search-container">
9       <form id="search-form">
10        <button id="search-btn" type="button">
11          <i class="fas fa-search"></i> Buscar perfil por RUT
12        </button>
13        <div class="search-input-container">
14          <input
15            type="text"
16            name="rut"
17            id="search-input"
18            placeholder="1234567-8"
19            required
20            pattern="^\d{7,8}-[0-9kk]$"
21            title="Formato esperado: 12345678-9"
22          />
23          <button type="submit" class="btn-send">
24            <i class="fas fa-arrow-right"></i>
25          </button>
26        </div>
27      </form>
28    </div>
29    <a href="/dashboard" class="navbar-item">
30      <i class="fas fa-tachometer-alt"></i> Dashboard
31    </a>
32    <a href="/register-admin" class="navbar-item">
33      <i class="fas fa-users"></i> Registrar admin.
34    </a>
35    <a href="/register-user" class="navbar-item">
36      <i class="fas fa-user-plus"></i> Registrar usuario
37    </a>
38    <form action="/logout" method="post" class="navbar-item">
39      <button type="submit" class="btn-logout">
40        <i class="fas fa-sign-out-alt"></i> Cerrar Sesión
41      </button>
42    </form>
43  </div>
44 </nav>
45 <script src="/static/js/navbar.js"></script>
```

Figura 50: Archivo navbar.html

8. navbar.js:

- a) Este código contiene la función que activa la visualización del navbar modificando su clase y por lo tanto su CSS, además de la lógica para hacer funcionar el boton que busca un rut específico:

```
1 document.addEventListener("DOMContentLoaded", function () {
2     // Manejo del menú hamburguesa
3     const hamburger = document.querySelector(".hamburger");
4     const menu = document.querySelector("#navbar-menu");
5
6     hamburger.addEventListener("click", function () {
7         const expanded = this.getAttribute("aria-expanded") === "true";
8         this.setAttribute("aria-expanded", !expanded);
9         menu.classList.toggle("active");
10    });
11
12    // Manejo de la búsqueda por RUT
13    const searchContainer = document.querySelector(".search-container");
14    const searchBtn = document.getElementById("search-btn");
15    const searchInput = document.getElementById("search-input");
16    const searchForm = document.getElementById("search-form");
17
18    // Contenedor para mostrar errores
19    const errorContainer = document.createElement("div");
20    errorContainer.id = "error-message";
21    errorContainer.style.color = "red";
22    errorContainer.style.fontSize = "14px";
23    errorContainer.style.fontFamily = "'Montserrat', sans-serif";
24    errorContainer.style.marginTop = "-5px"
25
26    searchContainer.insertAdjacentElement("afterend", errorContainer);
27
28
29    // Mostrar input de búsqueda al hacer clic en el botón
30    searchBtn.addEventListener("click", function () {
31        searchContainer.classList.add("active");
32        searchInput.focus();
33    });
34
35    // Cerrar la búsqueda si se hace clic fuera del contenedor
36    document.addEventListener("click", function (event) {
37        if (!searchContainer.contains(event.target)) {
38            searchContainer.classList.remove("active");
39        }
40    });
41});
```

Figura 51: Archivo navbar.js Parte I

- b) Este código maneja las validaciones del backend para mostrar si el rut existe o si no cumple con el formato:

```
42 // Manejo de la validación y solicitud al backend
43 searchForm.addEventListener("submit", function (event) {
44     event.preventDefault(); // Evita recargar la página
45     errorContainer.textContent = ""; // Borra errores previos
46
47     const rut = searchInput.value.trim();
48     if (!rut) {
49         errorContainer.textContent = "El campo RUT no puede estar vacío.";
50         return;
51     }
52
53     fetch(`/profile?rut=${encodeURIComponent(rut)}`, {
54         method: "GET",
55         headers: { "Accept": "application/json" },
56     })
57     .then(async (response) => {
58         const data = await response.json();
59         if (!response.ok) {
60             // Si la respuesta contiene errores, los mostramos
61             if (data.errors) {
62                 errorContainer.textContent = data.errors.map(err => err.message).join(" ");
63             } else {
64                 errorContainer.textContent = "Ocurrió un error inesperado.";
65             }
66         } else {
67             // Si no hay errores, redirige al perfil
68             window.location.href = `/profile?rut=${encodeURIComponent(rut)}`;
69         }
70     })
71     .catch((error) => {
72         console.error("Error al buscar el perfil:", error);
73         errorContainer.textContent = "Error al procesar la solicitud.";
74     });
75 });
76 });
77
```

Figura 52: Archivo navbar.js Parte II

9. dashboard.html:

- a) Se le dice a jinja2 que se utiliza base.html, se le agrega un título a la página y se agregan los componentes navbar y notify:

```
1 {% extends "base.html" %}
2 {% block title %}Dashboard{% endblock %}
3 {% block content %}
4 {% include "notify.html" %}
5 {% include "navbar.html" %}
6 <div class="dashboard-page">
7     <div class="sidebar-container">
8         <aside class="sidebar">
9             <h2>Filtros <i class="fas fa-filter"></i> </h2>
```

Figura 53: Archivo dashboard.html Parte I

b) Se pone un filtro para el año:

```
10 <div class="filter-group">
11   <label for="year-filter">Año: </label>
12   <input type="number" id="year-filter" min="2025" step="1" value="2025" onchange="fetchFilteredData()">
13 </div>
```

Figura 54: Archivo dashboard.html Parte II

c) Se pone un filtro para el mes:

```
14 <div class="filter-group">
15   <label for="month-filter">Mes: </label>
16   <select class="select-month" id="month-filter" onchange="fetchFilteredData()">
17     <option value="all">Todos </option>
18     <option value="1">Enero </option>
19     <option value="2">Febrero </option>
20     <option value="3">Marzo </option>
21     <option value="4">Abril </option>
22     <option value="5">Mayo </option>
23     <option value="6">Junio </option>
24     <option value="7">Julio </option>
25     <option value="8">Agosto </option>
26     <option value="9">Septiembre </option>
27     <option value="10">Octubre </option>
28     <option value="11">Noviembre </option>
29     <option value="12">Diciembre </option>
30   </select>
31 </div>
```

Figura 55: Archivo dashboard.html Parte III

d) Se pone un filtro para el día:

```
33 <div class="filter-group">
34   <label for="day-filter">Día: </label>
35   <input type="number" id="day-filter" min="1" max="31" step="1" value="" placeholder="Todos"
36     onchange="fetchFilteredData()">
37 </div>
```

Figura 56: Archivo dashboard.html Parte IV

e) Se pone un filtro para el motivo:

```
39 <div class="filter-group">
40   <label for="motivo-filter">Motivo:</label>
41   <select class="select-motivo" id="motivo-filter" onchange="fetchFilteredData()">
42     <option value="all">Todos</option>
43     <option value="asignatura">Asignatura</option>
44     <option value="asistencia tecnica">Asistencia Técnica</option>
45     <option value="investigacion">Investigación</option>
46     <option value="practica">Práctica Profesional</option>
47     <option value="trabajo de titulo">Trabajo de Título</option>
48     <option value="transferencia tecnologica">Transferencia Tecnológica</option>
49   </select>
50 </div>
```

Figura 57: Archivo dashboard.html Parte V

f) Se pone un filtro para el rut:

```
52 <div class="filter-group">
53   <label for="rut-filter">RUT:</label>
54   <input type="text" id="rut-filter" placeholder="Ej: 12345678-9" onchange="fetchFilteredData()">
55 </div>
```

Figura 58: Archivo dashboard.html Parte VI

g) Se pone un boton para resetear los filtros a un valor por defecto:

```
57 <button class="btn-reset" onclick="resetFilters()">
58   <i class="fas fa-eraser"></i> Limpiar filtros
59 </button>
```

Figura 59: Archivo dashboard.html Parte VII

h) Se pone un boton para mostrar/esconder el SIDEBAR:

```
60 </aside>
61 <button class="btn-filter">
62   <i class="fas fa-angle-double-left"></i>
63 </button>
64 </div>
```

Figura 60: Archivo dashboard.html Parte VIII

- i) Se pone una sección para mostrar la estadística del total de horas y total de usuarios que han utilizado el laboratorio:

```
66 <main class="main-content">
67   <h1 id="space-title">Utilización de CIM</h1>
68
69   <div class="stats-and-details">
70     <div class="stats-grid">
71       <div class="stat-card">
72         <h3>Total de Horas Utilizado</h3>
73         <p class="stat-value" id="total-horas">0</p>
74       </div>
75       <div class="stat-card">
76         <h3>Total de Usuarios Beneficiados</h3>
77         <p class="stat-value" id="total-usuarios">0</p>
78       </div>
79     </div>
```

Figura 61: Archivo dashboard.html Parte IX

- j) Se pone una sección para mostrar la tabla de registros desde el más reciente al más antiguo:

```
81   <div class="table-card">
82     <h3>Detalle de Actividades</h3>
83     <table id="details-table">
84       <thead>
85         <tr>
86           <th>Fecha</th>
87           <th>Motivo</th>
88           <th>Usuario</th>
89           <th>Verificado</th>
90           <th>Ingreso</th>
91           <th>Salida</th>
92         </tr>
93       </thead>
94       <tbody id="details-body">
95         <!-- Aquí se llenarán los datos dinámicamente -->
96       </tbody>
97     </table>
98
99     <!-- Contenedor de botones de paginación -->
100    <div id="pagination-container" class="pagination"></div>
101  </div>
102 </div>
```

Figura 62: Archivo dashboard.html Parte X

k) Se pone una sección para mostrar algunas gráficas:

```
104     <div class="charts-grid">
105         <div class="chart-card">
106             <h3>Utilización por Motivo</h3>
107             <canvas id="motivos-chart"></canvas>
108         </div>
109         <div class="chart-card">
110             <h3>Evolución de Utilización Mensual</h3>
111             <canvas id="mensual-chart"></canvas>
112         </div>
113         <div class="chart-card">
114             <h3>Uso de Espacio por Día</h3>
115             <canvas id="dia-chart"></canvas>
116         </div>
117     </div>
118 </main>
119 </div>
```

Figura 63: Archivo dashboard.html Parte XI

l) Se agregan los scripts y una función para mostrar el correcto registro de un usuario o administrador:

```
121 <!-- Scripts -->
122 <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
123 <script src="/static/js/charts.js"></script>
124 <script src="/static/js/data.js"></script>
125 <script src="/static/js/pagination.js"></script>
126 <script src="/static/js/sidebar.js"></script>
127 <script>
128     const urlParams = new URLSearchParams(window.location.search);
129     const message = urlParams.get('message');
130     if (message) {
131         alert(decodeURIComponent(message));
132     }
133 </script>
134
135 {% endblock %}
```

Figura 64: Archivo dashboard.html Parte XII

10. charts.js:

a) Etiquetas para los meses:

```
1 // Array de meses para reutilizar
2 const meses = [
3   "enero", "febrero", "marzo", "abril", "mayo", "junio",
4   "julio", "agosto", "septiembre", "octubre", "noviembre", "diciembre",
5 ];
```

Figura 65: Archivo charts.js Parte I

b) Configuración gráfica de barra por motivo:

```
7 // Gráfico de Utilización de Espacios
8 const motivosChartCtx = document
9   .getElementById("motivos-chart")
10  .getContext("2d");
11 const motivosChart = new Chart(motivosChartCtx, {
12   type: "bar",
13   data: {
14     labels: [
15       "asignatura",
16       "asistencia tecnica",
17       "investigacion",
18       "practica profesional",
19       "trabajo de titulo",
20       "transferencia tecnologica",
21     ],
22     datasets: [
23       {
24         label: "Suma de Horas",
25         data: Array(6).fill(0), // Inicialmente vacío
26         backgroundColor: "rgba(255, 206, 86, 0.7)",
27         borderColor: "rgba(255, 206, 86, 1)",
28         borderWidth: 1,
29       },
30     ],
31   },
32   options: {
33     responsive: true,
34     maintainAspectRatio: false,
35     plugins: {
36       legend: {
37         display: false,
38       },
39     },
40     scales: {
41       y: {
42         beginAtZero: true,
43       },
44     },
45   },
46 });
```

Figura 66: Archivo charts.js Parte II

c) Configuración gráfica de puntos evolución de utilización mensual:

```
48 // Gráfico de Evolución Mensual
49 const mensualChartCtx = document
50   .getElementById("mensual-chart")
51   .getContext("2d");
52 const mensualChart = new Chart(mensualChartCtx, {
53   type: "line",
54   data: {
55     labels: meses, // Usar el array de meses definido
56     datasets: [
57       {
58         label: "Cantidad de Horas",
59         data: Array(12).fill(0), // Inicialmente vacío
60         backgroundColor: "rgba(75, 192, 192, 0.2)",
61         borderColor: "rgba(75, 192, 192, 1)",
62         borderWidth: 2,
63         fill: true,
64       },
65     ],
66   },
67   options: {
68     responsive: true,
69     maintainAspectRatio: false,
70     plugins: {
71       legend: {
72         display: false,
73       },
74     },
75     scales: {
76       y: {
77         beginAtZero: true,
78       },
79     },
80   },
81 });
```

Figura 67: Archivo charts.js Parte III

d) Configuración gráfica de barra por uso de día:

```
83 // Gráfico de Uso de Espacio por Día
84 const diaChartCtx = document.getElementById("dia-chart").getContext("2d");
85 const diaChart = new Chart(diaChartCtx, {
86   type: "bar",
87   data: {
88     labels: ["lunes", "martes", "miércoles", "jueves", "viernes"], // No cambia
89     datasets: [
90       {
91         label: "Horas Acumuladas",
92         data: Array(5).fill(0), // Inicialmente vacío
93         backgroundColor: "rgba(255, 99, 132, 0.7)",
94         borderColor: "rgba(255, 99, 132, 1)",
95         borderWidth: 1,
96       },
97     ],
98   },
99   options: {
100     indexAxis: "y", // Hace que las barras sean horizontales
101     responsive: true,
102     maintainAspectRatio: false,
103     plugins: {
104       legend: {
105         display: false,
106       },
107     },
108     scales: {
109       x: {
110         beginAtZero: true,
111       },
112     },
113   },
114 });
```

Figura 68: Archivo charts.js Parte IV

11. data.js:

- a) Función que se encarga de obtener los valores de los filtros, crear la petición a la API y actualizar los datos:

```
1 // Función para obtener los datos filtrados desde el backend
2 async function fetchFilteredData() {
3   const year = document.getElementById("year-filter").value;
4   const month = document.getElementById("month-filter").value;
5   const day = document.getElementById("day-filter").value;
6   const motivo = document.getElementById("motivo-filter").value;
7   const rut = document.getElementById("rut-filter").value;
8
9   // Construir la URL con los parámetros de filtro
10  let url = `/api/dashboard?year=${year}`;
11  if (month !== "all") url += `&month=${month}`;
12  if (day !== "") url += `&day=${day}`;
13  if (motivo !== "all") url += `&motivo=${motivo}`;
14  if (rut !== "") url += `&rut=${rut}`;
15
16  try {
17    const response = await fetch(url);
18    const data = await response.json();
19    renderTable(data.registros); // Renderizar la tabla con los datos recibidos
20    updateStats(data.registros); // Actualizar estadísticas
21    paginateTable(5);
22    updateCharts(data.registros);
23  } catch (error) {
24    console.error("Error al obtener los datos:", error);
25  }
26 }
```

Figura 69: Archivo data.js Parte I

b) Función que se encarga de renderizar y formatear los datos de la tabla:

```
28 // Función para renderizar la tabla con los datos
29 function renderTable(data) {
30   const tbody = document.getElementById("details-body");
31   const fragment = document.createDocumentFragment(); // Crea un fragmento
32
33   data.forEach((item) => {
34     // Descomponer la fecha manualmente para evitar problemas de zona horaria
35     const [year, month, day] = item.fecha.split("-");
36     const fecha = `${day}-${month}-${year}`;
37     const verificado = item.verificado;
38
39     // Formatear hora a hh:mm
40     const horaIngreso = item.hora_ingreso
41       ? item.hora_ingreso.slice(0, 5)
42       : "N/A";
43     const horaSalida =
44       item.hora_salida && item.hora_salida !== "None"
45       ? item.hora_salida.slice(0, 5)
46       : "N/A";
47
48     // Formatear motivo a Start Case
49     const motivo = item.motivo
50       ? item.motivo
51         .split("_")
52         .map(
53           (word) => word.charAt(0).toUpperCase() + word.slice(1).toLowerCase()
54         )
55         .join(" ")
56       : "N/A";
57
58     // Agregar hipervínculo al RUT
59     // const rutLink = `<a href="#" class="rut-link" data-rut="${item.rut}">${item.rut}</a>`;
60     const rutLink = `<a href="/profile?rut=${encodeURIComponent(item.rut)}" class="rut-link">${item.rut}</a>`;
61
62     // Crear fila como nodo
63     const row = document.createElement("tr");
64     row.innerHTML = `
65       <td>${fecha}</td>
66       <td>${motivo}</td>
67       <td>${rutLink}</td>
68       <td>${verificado}</td>
69       <td>${horaIngreso}</td>
70       <td>${horaSalida}</td>
71     `;
72
73     // Añadir fila al fragmento
74     fragment.appendChild(row);
75   });
76
77   // Limpiar tbody y añadir el fragmento en una sola operación
78   tbody.innerHTML = "";
79   tbody.appendChild(fragment);
80 }
```

Figura 70: Archivo data.js Parte II

- c) Función que se encarga de calcular las estadísticas del total de usuarios y la cantidad de horas:

```
82 function updateStats(data) {
83   let totalHoras = 0;
84
85   // Crear un conjunto para contar usuarios únicos
86   const usuariosUnicos = new Set();
87
88   // Recorrer los datos para calcular las horas y contar los usuarios
89   data.forEach((item) => {
90     // Ignorar registros donde la hora_salida es null o None
91     if (item.hora_salida && item.hora_salida !== "None") {
92       // Convertir las horas a objetos Date
93       const horaIngreso = new Date(`1970-01-01T${item.hora_ingreso}`);
94       const horaSalida = new Date(`1970-01-01T${item.hora_salida}`);
95
96       // Calcular la diferencia en milisegundos y convertir a horas
97       const diferenciaHoras = (horaSalida - horaIngreso) / (1000 * 60 * 60);
98
99       // Sumar la diferencia al total de horas
100      totalHoras += diferenciaHoras;
101    }
102
103    // Agregar el RUT al conjunto de usuarios únicos
104    usuariosUnicos.add(item.rut);
105  });
106
107  // Mostrar los resultados en la página
108  document.getElementById("total-horas").textContent = totalHoras.toFixed(2);
109  document.getElementById("total-usuarios").textContent =
110    usuariosUnicos.size;
111 }
```

Figura 71: Archivo data.js Parte III

- d) Función que se encarga de actualizar la información de las gráficas, definición de etiquetas por temas de acentos:

```
113 function updateCharts(data) {
114   // Función para obtener el día de la semana
115   function getDayOfWeek(fecha) {
116     const diasSemana = ["domingo", "lunes", "martes", "miércoles", "jueves", "viernes", "sábado"];
117     const date = new Date(fecha);
118     return diasSemana[date.getUTCDay()]; // Devuelve el nombre del día en español
119   }
120
121   // Etiquetas internas (sin acentos, en minúsculas)
122   const motivosLabels = [
123     "asignatura",
124     "asistencia tecnica",
125     "investigacion",
126     "practica profesional",
127     "trabajo de titulo",
128     "transferencia tecnologica",
129   ];
130
131   const mesesLabels = [
132     "enero", "febrero", "marzo", "abril", "mayo", "junio",
133     "julio", "agosto", "septiembre", "octubre", "noviembre", "diciembre",
134   ];
135
136   const diasSemanaLabels = ["lunes", "martes", "miércoles", "jueves", "viernes"];
137
138   // Etiquetas para visualización (formateadas con Start Case y acentos)
139   const motivosDisplayLabels = [
140     "Asignatura",
141     "Asistencia Técnica",
142     "Investigación",
143     "Práctica Profesional",
144     "Trabajo de Título",
145     "Transferencia Tecnológica",
146   ];
```

Figura 72: Archivo data.js Parte IV

- e) Función que se encarga de actualizar las gráficas, procesando los datos y insertandolos en las gráficas:

```
148 const mesesDisplayLabels = [  
149     "Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio",  
150     "Julio", "Agosto", "Septiembre", "Octubre", "Noviembre", "Diciembre",  
151 ];  
152  
153 const diasSemanaDisplayLabels = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes"];  
154  
155 // Datos iniciales  
156 const motivosData = Array(motivosLabels.length).fill(0);  
157 const mensualData = Array(mesesLabels.length).fill(0);  
158 const diaData = Array(diasSemanaLabels.length).fill(0);  
159  
160 // Procesar los datos  
161 data.forEach((item) => {  
162     if (item.hora_ingreso && item.hora_salida && item.hora_salida !== "None") {  
163         // Calcular la diferencia de horas  
164         const horaIngreso = new Date(`1970-01-01T${item.hora_ingreso}`);  
165         const horaSalida = new Date(`1970-01-01T${item.hora_salida}`);  
166         const diferenciaHoras = (horaSalida - horaIngreso) / (1000 * 60 * 60);  
167  
168         // Procesar datos por motivo  
169         const motivoIndex = motivosLabels.indexOf(item.motivo);  
170         if (motivoIndex ≥ 0) {  
171             motivosData[motivoIndex] += diferenciaHoras;  
172         }  
173  
174         // Procesar datos por mes  
175         const monthIndex = parseInt(item.fecha.split("-")[1], 10) - 1;  
176         if (monthIndex ≥ 0 && monthIndex < 12) {  
177             mensualData[monthIndex] += diferenciaHoras;  
178         }  
179  
180         // Procesar datos por día  
181         const diaSemana = getDayOfWeek(item.fecha);  
182         const dayIndex = diasSemanaLabels.indexOf(diaSemana.toLowerCase());  
183         if (dayIndex ≥ 0) {  
184             diaData[dayIndex] += diferenciaHoras;  
185         }  
186     }  
187 });  
188  
189 // Actualizar datos de los gráficos  
190 motivosChart.data.labels = motivosDisplayLabels; // Etiquetas visuales  
191 motivosChart.data.datasets[0].data = motivosData;  
192 motivosChart.update();  
193  
194 mensualChart.data.labels = mesesDisplayLabels; // Etiquetas visuales  
195 mensualChart.data.datasets[0].data = mensualData;  
196 mensualChart.update();  
197  
198 diaChart.data.labels = diasSemanaDisplayLabels; // Etiquetas visuales  
199 diaChart.data.datasets[0].data = diaData;  
200 diaChart.update();  
201 }
```

Figura 73: Archivo data.js Parte V

f) Función para resetear los valores de los filtros:

```
204 function resetFilters() {  
205     const currentYear = new Date().getFullYear();  
206  
207     document.getElementById("year-filter").value = currentYear;  
208     document.getElementById("month-filter").value = "all";  
209     document.getElementById("day-filter").value = "";  
210     document.getElementById("motivo-filter").value = "all";  
211     document.getElementById("rut-filter").value = "";  
212  
213     fetchFilteredData();  
214 }
```

Figura 74: Archivo data.js Parte VI

g) Código que verifica que solo los rut con href puedan tener una referencia:

```
216 // Escuchar los clics en los enlaces de la clase "rut-link"  
217 document.addEventListener("click", (event) => {  
218     const target = event.target;  
219  
220     if (target.classList.contains("rut-link")) {  
221         // Verifica si tiene el atributo href  
222         const url = target.getAttribute("href");  
223  
224         if (url) {  
225             // Permitir la redirección normal  
226             return;  
227         }  
228  
229         // Si no tiene href, entonces es el caso donde filtras los datos  
230         event.preventDefault();  
231  
232         const rut = target.getAttribute("data-rut");  
233  
234         if (rut) {  
235             document.getElementById("rut-filter").value = rut;  
236             fetchFilteredData();  
237         }  
238     }  
239 });
```

Figura 75: Archivo data.js Parte VII

- h) Se vinculan los filtros a la función para que cada vez que ocurra algún cambio dentro de sus valores se active la función:

```
242 // Vincular los eventos de los filtros para actualizar los datos
243 document
244   .getElementById("year-filter")
245   .addEventListener("change", fetchFilteredData);
246 document
247   .getElementById("month-filter")
248   .addEventListener("change", fetchFilteredData);
249 document
250   .getElementById("day-filter")
251   .addEventListener("change", fetchFilteredData);
252 document
253   .getElementById("motivo-filter")
254   .addEventListener("change", fetchFilteredData);
255 document
256   .getElementById("rut-filter")
257   .addEventListener("input", fetchFilteredData);
```

Figura 76: Archivo data.js Parte VIII

- i) Cargamos los datos:

```
258 // Cargar los datos al inicio
259 document.addEventListener("DOMContentLoaded", fetchFilteredData);
```

Figura 77: Archivo data.js Parte IX

12. pagination.js:

- a) Se muestra la tabla con 5 filas de datos:

```
1 document.addEventListener("DOMContentLoaded", function () {
2   paginateTable(5); // Muestra 5 registros por página
3 });
```

Figura 78: Archivo pagination.js Parte I

- b) Función que renderiza inicialmente la tabla calculando la cantidad de filas a mostrar:

```
5 function paginateTable(rowsPerPage) {
6   const tbody = document.getElementById("details-body");
7   const rows = Array.from(tbody.querySelectorAll("tr"));
8   const paginationContainer = document.getElementById("pagination-container");
9
10  let currentPage = 1;
11  const totalPages = Math.ceil(rows.length / rowsPerPage);
12
13  function renderTable(page) {
14    tbody.innerHTML = "";
15    const start = (page - 1) * rowsPerPage;
16    const end = start + rowsPerPage;
17
18    rows.slice(start, end).forEach((row) => tbody.appendChild(row));
19  }
```

Figura 79: Archivo pagination.js Parte II

- c) Función que renderiza los botones de la paginación, en este caso el botón para Primera Página:

```
21 function renderPagination() {
22   paginationContainer.innerHTML = "";
23
24   // Botón "Primera Página"
25   const firstButton = document.createElement("button");
26   firstButton.textContent = "Inicio";
27   firstButton.disabled = currentPage === 1;
28   firstButton.addEventListener("click", () => {
29     currentPage = 1;
30     renderTable(currentPage);
31     renderPagination();
32   });
33   paginationContainer.appendChild(firstButton);
```

Figura 80: Archivo pagination.js Parte III

d) Se renderiza el botón Anterior:

```
35 // Botón "Anterior"
36 const prevButton = document.createElement("button");
37 prevButton.textContent = "Anterior";
38 prevButton.disabled = currentPage === 1;
39 prevButton.addEventListener("click", () => {
40     currentPage = Math.max(1, currentPage - 1);
41     renderTable(currentPage);
42     renderPagination();
43 });
44 paginationContainer.appendChild(prevButton);
```

Figura 81: Archivo pagination.js Parte IV

e) Se renderiza el input para mostrar la paginación actual o elegir una:

```
46 // Input para escribir página específica
47 const pageInput = document.createElement("input");
48 pageInput.type = "number";
49 pageInput.min = 1;
50 pageInput.max = totalPages;
51 pageInput.value = currentPage;
52 pageInput.addEventListener("change", () => {
53     const page = parseInt(pageInput.value);
54     if (page ≥ 1 && page ≤ totalPages) {
55         currentPage = page;
56         renderTable(currentPage);
57         renderPagination();
58     } else {
59         pageInput.value = currentPage;
60     }
61 });
62 paginationContainer.appendChild(pageInput);
```

Figura 82: Archivo pagination.js Parte V

f) Se renderiza el botón Siguiente:

```
64 // Botón "Siguiente"
65 const nextButton = document.createElement("button");
66 nextButton.textContent = "Siguiente";
67 nextButton.disabled = currentPage === totalPages;
68 nextButton.addEventListener("click", () => {
69     currentPage = Math.min(totalPages, currentPage + 1);
70     renderTable(currentPage);
71     renderPagination();
72 });
73 paginationContainer.appendChild(nextButton);
```

Figura 83: Archivo pagination.js Parte VI

g) Se renderiza el botón Última Página:

```
75 // Botón "Última Página"
76 const lastButton = document.createElement("button");
77 lastButton.textContent = "Final";
78 lastButton.disabled = currentPage === totalPages;
79 lastButton.addEventListener("click", () => {
80     currentPage = totalPages;
81     renderTable(currentPage);
82     renderPagination();
83 });
84 paginationContainer.appendChild(lastButton);
85 }
86
87 renderTable(currentPage);
88 renderPagination();
89 }
```

Figura 84: Archivo pagination.js Parte VII

13. sidebar.js:

a) Función para mostrar/esconder el SIDEBAR:

```
1 document.addEventListener("DOMContentLoaded", () => {
2   const btnFilter = document.querySelector(".btn-filter");
3   const sidebarContainer = document.querySelector(".sidebar-container");
4   const sidebar = document.querySelector(".sidebar");
5   const mainContent = document.querySelector(".main-content");
6
7   btnFilter.addEventListener("click", () => {
8     // Alternar clases en el contenedor (no en el sidebar)
9     sidebarContainer.classList.toggle("hidden");
10    sidebar.classList.toggle("hidden");
11    mainContent.classList.toggle("expanded");
12
13    // Cambiar el ícono
14    const icon = btnFilter.querySelector("i");
15    icon.classList.toggle("fa-angle-double-left");
16    icon.classList.toggle("fa-angle-double-right");
17  });
18 });
```

Figura 85: Archivo sidebar.js

5.2. Software registro de asistencia

5.2.1. Librerías

6. Repositorio de GitHub

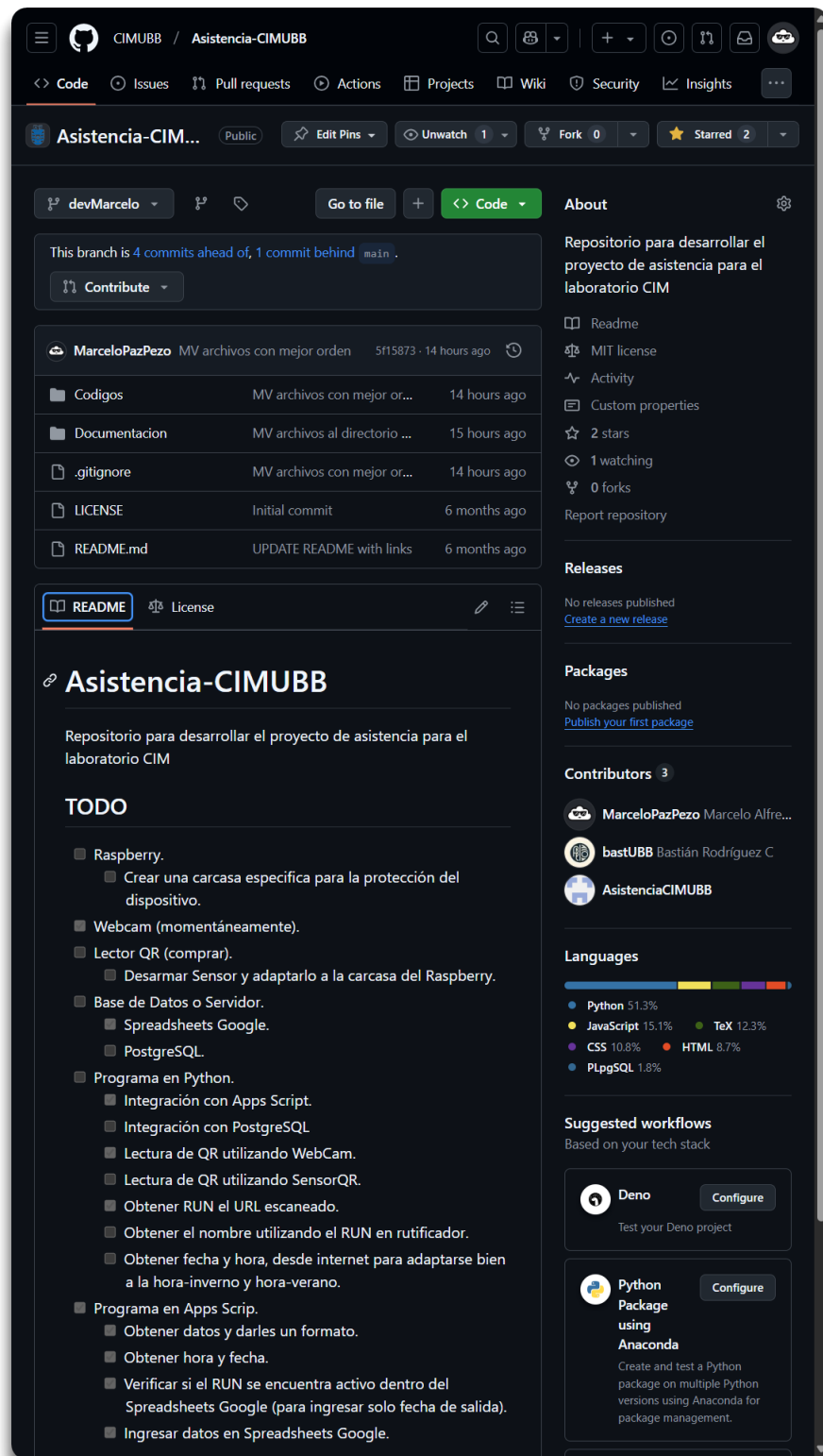


Figura 86: Repositorio del proyecto [5]



Referencias

- [1] Raspberry Pi 4 Model B 8 GB RAM <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>
- [2] Logitech HD Webcam C270 https://www.logitech.com/es-roam/products/webcams/c270-hd-webcam.960-000947.html?srsltid=AfmB0oqdKlo4lP1JlkMP6cvLw_Y6Kj5oKXIW-8gIEuyMUwLZRGhJMIwS
- [3] Documentación sensor QR GM811 <https://robu.in/wp-content/uploads/2024/08/GM811.pdf>
- [4] Generate Random Strings and Numbers <https://www.browserling.com/tools/random-string>
- [5] Asistencia-CIMUBB <https://github.com/CIMUBB/Asistencia-CIMUBB>